

Универзитет у Крагујевцу

Bogdan Babaev

Forecasting Time-Varying Intermarket
Dependencies Between
Cryptocurrencies and Conventional
Assets Using Machine Learning

Мастер рад

Крагујевац, 2026.

Универзитет у Крагујевцу



Назив студијског програма: ВЕШТАЧКА ИНТЕЛИГЕНЦИЈА

Ниво студија: Мастер академске студије

Предмет: Напредно машинско учење

Број индекса: 1ВИ/2023

Bogdan Babaev

Forecasting Time-Varying Intermarket Dependencies Between Cryptocurrencies and Conventional Assets Using Machine Learning

Мастер рад

Комисија за преглед и одбрану:

1. проф. др Владимир М. Миловановић - ментор
2. др Александар Пеулић, редовни професор
3. др Јасна Радуловић, редовни професор
4. др Милан Чабаркапа, доцент
5. др Иван Крстић, ванредни професор

Датум одбране: _____

Оцена: _____

У оквиру овог мастер рада кандидат треба да истражи могућност прогнозирања временски-променљивих међутржишних зависности између криптовалута и конвенционалних финансијских актива применом модела машинског учења. У склопу тога кандидат треба да:

1. прикупи историјске дневне цене за Bitcoin и традиционалне активе (акцијске индексе, фондове племенитих метала, индекс америчког долара, Ethereum) и изврши одговарајућу предобраду и синхронизацију скупа података;
2. развије репродуцибилан рачунарски поступак за израчунавање временски-променљивих корелационих циљева помоћу клизних прозора различитих дужина и конструкцију предиктивних особина заснованих на инерцији зависности, волатилности и повратима;
3. реализује ванузорачну евалуацију и поређење неколико модела машинског учења (AR, ElasticNet, Ridge, Random Forest, GBM, XGBoost) са DCC-GARCH(1,1) економетријским репером применом процедуре проклизавајућег прозора и Diebold-Mariano теста статистичке значајности;
4. уведе практични сигнални слој за рано детектовање стресних дана на тржишту традиционалних актива на основу динамике криптовалута и прогноза зависности, и вреднује га метрикама класификационе тачности прикладним за неуравнотежен скуп класа.

Препоручена литература:

- [1] Robert F. Engle, „Dynamic Conditional Correlation: A Simple Class of Multivariate Generalized Autoregressive Conditional Heteroskedasticity Models”, у: *Journal of Business & Economic Statistics* 20.3 (2002.), стр. 339–350.
- [2] Francis X. Diebold и Roberto S. Mariano, „Comparing Predictive Accuracy”, у: *Journal of Business & Economic Statistics* 13.3 (1995.), стр. 253–263.
- [3] Hui Zou и Trevor Hastie, „Regularization and Variable Selection via the Elastic Net”, у: *Journal of the Royal Statistical Society: Series B* 67.2 (2005.), стр. 301–320.
- [4] Jerome H. Friedman, „Greedy Function Approximation: A Gradient Boosting Machine”, у: *The Annals of Statistics* 29.5 (2001.), стр. 1189–1232.
- [5] Tianqi Chen и Carlos Guestrin, „XGBoost: A Scalable Tree Boosting System”, у: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016., стр. 785–794.

Крагујевац, 19. 5. 2026.

Ментор:

Др Владимир М. Миловановић,
ванредни професор

Abstract

This thesis investigates whether information contained in cryptocurrency dynamics can be used to forecast time-varying dependency structures with conventional assets and, in a second step, to derive an early warning signal for investors. Bitcoin is treated as the base cryptocurrency, while the conventional asset universe includes equity indices, precious-metal exchange-traded funds, the U.S. dollar index, and Ethereum as an additional crypto reference. A reproducible machine-learning pipeline is developed for data acquisition, return construction, rolling-correlation target generation, feature engineering, walk-forward model evaluation, and benchmarking against a leakage-safe DCC-GARCH specification. The empirical results show that intermarket dependency is forecastable out of sample across all asset pairs and window lengths, but that most predictive power originates from persistence in the dependency series itself. Ridge regression, AR(1), and the HAR model form a near-indistinguishable top cluster, each exploiting serial persistence through different mechanisms while converging to essentially the same out-of-sample accuracy; Ridge achieves the lowest average RMSE of 0.0656. All ten models substantially outperform the DCC-GARCH benchmark. An investor-oriented signal layer based on crypto features and dependency forecasts achieves moderate but consistent performance and is best interpreted as a risk-management overlay rather than a standalone market-timing engine.

Keywords: cryptocurrencies, Bitcoin, intermarket dependency, correlation, machine learning, DCC-GARCH, forecasting, risk management.

Резиме

У овом мастер раду разматра се проблем прогнозирања променљивих међутржишних зависности између криптовалута и традиционалних финансијских актива. У фокусу је Bitcoin као основна криптовалута, док скуп традиционалних актива обухвата акцијске индексе, фондове везане за племените метале и индекс америчког долара. Развијен је репродуцибилан рачунарски поступак који покрива преузимање података, израчунавање логаритамских приноса, формирање временски променљивих корелационих циљева, конструисање особина, ванузорачну процену више модела машинског учења и поређење са DCC-GARCH економетријским репером. Поред основног задатка прогнозирања зависности, уведен је и практични слој за формирање раног сигнала ризика који треба да упозори инвеститора на стресне дане на тржишту традиционалних актива. Резултати показују да је временски променљива међутржишна зависност ванузорачно предвидива, при чему Ridge регресија, AR(1) и HAR модел формирају практично неразлучиву групу на врху рангирања — свако од њих користи аутокорелациону структуру циља на другачији начин, а Ridge постиже најнижи просечни RMSE од 0.0656. Сви модели значајно надмашују DCC-GARCH репер. Практични сигнални слој даје умерене резултате и има смисла пре свега као додатни филтар ризика, а не као самосталан механизам за свакодневно одређивање смера тржишта.

Кључне речи: криптовалуте, Bitcoin, међутржишне зависности, корелација, машинско учење, DCC-GARCH, управљање ризиком.

Contents

List of Abbreviations	9
1 Introduction	10
1.1 Motivation and research context	10
1.2 Research questions	10
1.3 Contributions of the thesis	10
1.4 Thesis outline	11
2 Theoretical Background and Literature Review	12
2.1 Why intermarket dependency matters	12
2.2 Cryptocurrencies and conventional assets	12
2.3 Rolling correlation as a forecasting target	12
2.4 Econometric benchmark: DCC-GARCH	13
2.5 Machine-learning approaches to forecasting	14
2.5.1 The HAR model and persistence-based forecasting	14
2.5.2 Regularized linear models	15
2.5.3 Tree-based ensemble models	15
2.6 Walk-forward evaluation and information leakage	15
2.7 Model comparison and the Diebold–Mariano test	16
2.8 From dependency forecasting to investor signaling	16
2.9 Positioning of the present thesis	17
3 Methodology	18
3.1 Research design	18
3.2 Asset universe and data source	18
3.3 Return construction and preprocessing	20
3.4 Target construction: rolling dependency	20
3.5 Feature engineering	22
3.6 Forecasting models	23
3.6.1 Baseline and autoregressive models	23
3.6.2 Regularized linear models	24
3.6.3 Tree ensembles and adaptive ensemble	24
3.6.4 DCC-GARCH benchmark	25
3.7 Investor signal layer	25
3.8 Walk-forward estimation framework	25
3.9 Evaluation metrics	26
3.10 Model comparison methodology	26
3.11 Implementation and reproducibility	26
3.12 Methodological expectations	26
4 Empirical Results	28
4.1 Overview of the empirical output	28
4.2 Average dependency-forecasting performance	28
4.3 Representative forecast example	29
4.4 Forecasting performance by asset class	30
4.5 The persistence dominance result	30
4.6 Comparison with DCC-GARCH	31

4.7	Error-profile diagnostics	32
4.8	Investor signal layer: average results	32
4.9	Best signal configurations	33
4.10	Practical interpretation of signal-layer performance	34
4.11	Sensitivity to rolling window length	34
4.12	Summary of empirical findings	35
5	Discussion	36
5.1	What the results actually say	36
5.2	Why simple models dominate, and what that actually means	36
5.3	ML versus DCC-GARCH: a fairer comparison	36
5.4	The investor signal: what works and what does not	37
5.5	Methodological choices that mattered	37
5.6	What the results do not say	38
5.7	Iterative model development: what changed across versions	38
5.8	Limitations, honestly stated	40
6	Conclusion	41
A	Appendix A: Data Description and Additional Background	42
A.1	Descriptive role of the asset universe	42
A.2	Visual overview of the dataset	42
A.3	Dataset validation and quality diagnostics	45
A.4	Fisher-z transformation of the correlation target	47
A.5	Why multiple rolling windows are used	48
A.6	Additional implementation remarks	48
A.7	Summary	48
B	Appendix B: Supplementary Forecasting Results	49
B.1	Model-comparison figures	49
B.2	Hyperparameter and feature diagnostics	50
B.3	Diagnostic plots for the representative equity pair	51
B.4	Sensitivity by asset pair	52
B.5	Interpretation	54
C	Appendix C: Supplementary Signal-Layer Results	55
C.1	Signal figures by asset	55
C.2	Cross-market warning interpretation	56
C.3	Summary table	57
D	Appendix D: Reproducibility and Code Structure	59
D.1	Project organization	59
D.2	Main implementation modules	59
D.3	Analysis notebooks	60
D.4	Execution flow	61
D.5	Configuration	62
D.6	Verification performed during the project cleanup	62
D.7	How to rebuild the thesis artifacts	63
D.8	Final reproducibility note	64

E	Appendix E: Additional Statistical Tests	65
E.1	Mincer-Zarnowitz Forecast Efficiency Test	65
E.2	Residual Diagnostics: Ljung-Box and ARCH LM Tests	66
E.3	Harvey-Leybourne-Newbold Corrected Diebold-Mariano Test	67
E.4	Automated Validation Suite	68
F	Appendix F: Market-Event Case Studies	70
	Објављени радови	82

List of Figures

1	Normalized price evolution of all seven assets over the full sample period (2017–2026). Series are scaled to a common base of 1 at the start date to allow visual comparison of relative performance.	19
2	Thirty-day rolling volatility of log-returns across the asset universe. Bitcoin and Ethereum exhibit substantially higher and more episodic volatility than conventional assets, motivating a dynamic rather than static treatment of cross-asset dependency.	19
3	Full-sample Pearson correlation matrix of daily log-returns. The low static correlations between Bitcoin and most conventional assets motivate studying the time-varying structure rather than relying on a single full-sample estimate.	20
4	Rolling dependency between Bitcoin and each of the five conventional assets across all four window lengths (14, 30, 60, 90 days). The Fisher-transformed series illustrates the strong time-variation and persistence that motivate the forecasting framework.	22
5	Observed and predicted Fisher-transformed dependency for the Bitcoin–S&P 500 pair at the 30-day rolling window. Forecast series are generated under strict walk-forward evaluation with no use of future information. . .	29
6	Diebold–Mariano test results comparing Ridge (best overall RMSE across all models) against DCC-GARCH across all asset-pair and window-length combinations. Darker cells indicate stronger and more statistically significant evidence of Ridge superiority.	31
7	Rolling out-of-sample RMSE over time for the Bitcoin–S&P 500 dependency experiment. Elevated error episodes correspond to periods of abrupt regime transition in the dependency process.	32
8	Investor stress-warning signal for the Bitcoin–S&P 500 pair at the 30-day window. The upper panel displays the predicted stress probability from the logistic classifier together with the decision threshold; the lower panel marks flagged and missed stress events.	34
9	Normalized prices of the assets included in the study.	43
10	Rolling volatility across the asset universe.	43
11	Static correlation heatmap of the return series.	44
12	Rolling correlation overview for the asset universe.	45
13	Observation coverage window per ticker. All assets share an identical aligned date range after the ETH-USD availability cutoff in November 2017. . .	46
14	Log-return series with extreme observations ($ z > 5\sigma$) highlighted in red. Crypto assets exhibit sporadic but large outliers; conventional assets are more contained but not outlier-free.	47
15	Distribution of the 30-day rolling correlation before and after Fisher-z transformation for the Bitcoin–S&P 500 pair. The transformation improves normality and removes boundary compression.	47
16	Average RMSE comparison across model families.	49
17	Model R^2 comparison for the 30-day window.	49
18	Improvement relative to the naive baseline.	50
19	Grid-search cross-validation RMSE for the representative Bitcoin–S&P 500 forecasting task.	50

20	Regularization path for the Ridge-based forecasting specification.	50
21	Readable feature-importance view for the Random Forest model.	51
22	Window sensitivity of the XGBoost-based forecasting layer.	51
23	Forecast scatter diagnostic for the Bitcoin–S&P 500 dependency experiment.	51
24	Forecast error distribution for the representative Bitcoin–S&P 500 depen- dency experiment.	52
25	Sensitivity analysis for the Bitcoin–NASDAQ pair.	52
26	Sensitivity analysis for the Bitcoin–gold pair.	52
27	Sensitivity analysis for the Bitcoin–silver pair.	53
28	Sensitivity analysis for the Bitcoin–UUP pair.	53
29	Sensitivity analysis for the Bitcoin–Ethereum pair.	53
30	Signal-layer output for the Bitcoin–NASDAQ relation at the 30-day window.	55
31	Signal-layer output for the Bitcoin–gold relation at the 30-day window. . .	55
32	Signal-layer output for the Bitcoin–silver relation at the 30-day window. . .	56
33	Signal-layer output for the Bitcoin–UUP relation at the 30-day window. . .	56
34	Supplementary S&P 500 warning signal at the 14-day window.	57
35	Supplementary S&P 500 warning signal at the 60-day window.	57
36	Supplementary S&P 500 warning signal at the 90-day window.	58
37	Supplementary NASDAQ warning signal at the 14-day window.	58
38	Supplementary NASDAQ warning signal at the 60-day window.	58
39	Master timeline 2018–2026: Bitcoin price, conventional-asset prices, and the 30-day rolling BTC–S&P 500 correlation, with all eleven landmark events marked. Red dotted lines = crash/shock events; green = rallies and all-time highs; blue = structural divergences; purple = institutional milestones.	70
40	Correlation regime map. Left: post-event BTC correlation (30-day aver- age) with each conventional asset. Right: change in correlation relative to the 30-day pre-event average. Red/blue = strong positive/negative shifts. .	71
41	BTC vs S&P 500 daily returns around each event (± 30 days). Grey points: pre-event; coloured points: post-event (colour matches event type). The correlation coefficient ρ annotates each panel.	72
42	Event regime map: BTC 7-day return at the event versus the change in the 30-day rolling BTC–S&P 500 correlation. Events in the upper-left quadrant correspond to macro risk-off crashes; events in the lower-right correspond to crypto-specific rallies that decoupled from equities.	73
43	Event 1 — COVID Black Thursday (12 March 2020). Bitcoin fell $\sim 50\%$ in 24 hours, sharply outpacing equity losses. The 30-day BTC– S&P 500 correlation spiked from near zero to above 0.70, demonstrating that liquidity-driven sell-offs erase diversification benefits. Source: Reuters	73
44	Event 2 — Institutional Adoption Wave (Q4 2020). MicroStrategy, Square, and PayPal announcements drove Bitcoin from \$10 K to \$20 K+ while equities were flat. Correlation fell back toward zero, confirming that the March 2020 co-crash was a liquidity event, not a structural shift. Source: The Guardian	74

45	Event 3 — China Mining Ban and Musk Tweets (May 2021). Bitcoin fell $\sim 40\%$ in one week; the S&P 500 barely moved (-1.5%). This textbook crypto-idiosyncratic shock produced only a brief, transitory correlation spike that quickly normalised — a sharp contrast to macro-driven events. Source: BBC News	74
46	Event 4 — Bitcoin ATH \$69 000 (10 November 2021). At the 2021 peak the 30-day BTC–NASDAQ correlation reached ~ 0.70 , reflecting shared risk-appetite drivers with technology stocks. This elevated correlation marked the onset of the 2022 bear market, as Federal Reserve rate hikes simultaneously depressed both assets. Source: BBC News	75
47	Event 5 — Luna/Terra Collapse (9–13 May 2022). The \$60B algorithmic stablecoin implosion drove Bitcoin down 25% while the S&P 500 fell only 5% on unrelated macro factors (Fed hikes). Gold was essentially flat, confirming that crypto contagion channels do not reliably extend to traditional safe-haven assets. Source: The Guardian	75
48	Event 6 — FTX Collapse (8–11 November 2022). SBF’s exchange filed for bankruptcy; Bitcoin fell 26% over 72 hours. Paradoxically the S&P 500 <i>rose</i> 5% the same week on a positive CPI print — producing a brief but striking negative BTC–equity correlation. Source: BBC News	76
49	Event 7 — SVB Banking Crisis (10 March 2023). Silicon Valley Bank’s failure triggered a 30% Bitcoin rally as investors sought alternatives to the banking system. Both Bitcoin and Gold rose while equities fell — one of the rare episodes in the sample where the 30-day BTC–S&P 500 correlation flipped to strongly negative. Source: The New York Times	76
50	Event 8 — SEC Approves Spot Bitcoin ETFs (10 January 2024). After a decade of rejections, BlackRock IBIT and Fidelity FBTC were approved. Bitcoin fell 15% on classic “buy the rumour, sell the news” behaviour. The structural significance: Bitcoin became accessible to US institutional investors via regulated wrappers for the first time. Source: CNBC	77
51	Event 9 — Bitcoin ATH \$73 000 (5 March 2024). Massive ETF inflows (BlackRock IBIT absorbed \$10B in 7 weeks) drove Bitcoin to a new all-time high. Both NASDAQ and Bitcoin reached ATHs in the same week; the 30-day BTC–NASDAQ correlation reached 0.65, confirming that institutional channels create sustained co-movement with growth equities. Source: CoinDesk	77
52	Event 10 — Trump Election Victory (6 November 2024). Bitcoin rose +35% in two weeks on Trump’s explicit pro-crypto platform (proposed BTC strategic reserve, light-touch regulation). Both crypto and equities rallied, producing a positive correlation episode driven by a shared political catalyst. Source: Reuters	78
53	Event 11 — “Liberation Day” Tariff Shock (2 April 2025). Sweeping tariffs on 185 countries caused the S&P 500 to fall 10% in four trading days — its fastest drop since March 2020. Bitcoin fell 18% alongside equities; Gold surged to above \$3 200/oz. This episode produced the highest single-event BTC–S&P 500 correlation spike in the dataset ($\rho > 0.80$) and illustrates most clearly that Bitcoin behaves as a risk asset, not a safe haven, during macro shocks. Source: Reuters	78

List of Tables

1	Asset universe used in the thesis.	18
2	Main groups of predictors used in the dependency-forecasting layer.	23
3	Average out-of-sample forecasting performance across all dependency experiments.	28
4	Average investor-signal performance by classifier family, aggregated across all experiments.	33
5	Data quality summary per ticker	46
6	Average signal-layer performance by classifier family.	57
7	Mincer-Zarnowitz forecast efficiency: percentage of 24 experiments passing the joint F -test ($p \geq 0.05$) and average coefficient estimates.	65
8	Mincer-Zarnowitz test: BTC-USD vs S&P 500, $w = 30$. F -statistic tests $H_0: \alpha = 0, \beta = 1$; p_{MZ} is the associated p -value.	66
9	Residual diagnostics: percentage of 24 experiments passing the Ljung-Box autocorrelation test and the ARCH LM heteroscedasticity test at $\alpha = 0.05$	67
10	HLN-corrected DM test: ML models vs DCC-GARCH benchmark, BTC-USD vs S&P 500, $w = 30$. DM: original statistic (normal); DM_{HLN} : corrected statistic (t_{n-1}). All models beat DCC-GARCH at $p < 0.001$ under both tests.	68
11	Automated validation suite results	69

List of Abbreviations

Abbreviation	Full form
AR1	First-order autoregressive model
AUC	Area under the ROC curve
BTC	Bitcoin (BTC-USD)
CSV	Comma-separated values
DCC	Dynamic Conditional Correlation
DCC-GARCH	Dynamic Conditional Correlation GARCH(1,1)
DM	Diebold–Mariano (test)
EDA	Exploratory Data Analysis
ElasticNet	Elastic Net regularized regression
ETF	Exchange-Traded Fund
ETH	Ethereum (ETH-USD)
F1	F1 score (harmonic mean of precision and recall)
GARCH	Generalized Autoregressive Conditional Heteroskedasticity
GBM	Gradient Boosting Machine
GLD	SPDR Gold Shares ETF (GLD)
GPU	Graphics Processing Unit
GSPC	S&P 500 index (^GSPC)
HAR	Heterogeneous Autoregressive (model)
IXIC	NASDAQ Composite index (^IXIC)
MAE	Mean Absolute Error
ML	Machine Learning
NASDAQ	National Association of Securities Dealers Automated Quotations
OOS	Out-of-sample
RF	Random Forest
RMSE	Root Mean Squared Error
ROC	Receiver Operating Characteristic
S&P 500	Standard & Poor’s 500 index
SLV	iShares Silver Trust ETF (SLV)
UUP	Invesco DB US Dollar Index Bullish Fund (UUP)
XGB	XGBoost (Extreme Gradient Boosting)

1 Introduction

1.1 Motivation and research context

Over the last decade, Bitcoin went from a curiosity on niche internet forums to an asset class that institutional investors actually worry about. That shift has a concrete consequence for risk management: when a previously isolated market grows large enough to attract serious capital, its price swings start to matter for portfolios that never held a single satoshi. The question is not whether Bitcoin now influences other markets, but *how* and *when* that influence changes over time.

This thesis focuses on *time-varying intermarket dependency*—the evolving statistical relationship between Bitcoin and conventional assets like equity indices, gold, silver, and the dollar. Rather than asking whether Bitcoin and the S&P 500 are correlated on average, the question is whether that correlation is predictable at all, and whether cryptocurrency price dynamics contain any early-warning signal for stress in traditional markets.

The practical angle matters. A risk manager does not need a crystal-ball return forecast to get value from this kind of analysis. What matters is recognising when co-movement patterns are changing, when a market regime is shifting from decorrelated to tightly coupled, or when the probability of a bad day on conventional markets is quietly rising. Even a modest, statistically honest signal of that kind can be useful—as long as you know what it can and cannot do.

1.2 Research questions

The thesis is structured around two connected layers. The first is a forecasting problem: can rolling correlation between Bitcoin and a conventional asset be predicted one step ahead, and how do machine-learning models compare to a classical econometric benchmark under strict out-of-sample evaluation? The second is an applied question: can those forecasts be turned into a practical warning signal for investors?

More specifically, the four research questions addressed are:

1. Is the rolling dependency between Bitcoin and conventional assets forecastable out of sample?
2. Do machine-learning models outperform a leakage-safe DCC-GARCH benchmark?
3. Do machine-learning models add value beyond simple persistence-based baselines such as AR1?
4. Can crypto-derived dependency forecasts support a practically useful stress-warning signal for conventional markets?

1.3 Contributions of the thesis

To address these questions, a reproducible Python pipeline was built from scratch. It downloads price data, constructs log-return series, computes rolling correlations across multiple window lengths, engineers lagged and volatility-based features, and evaluates a range of models within a walk-forward experimental design. The model set includes two naive baselines (random-walk and AR(1)), a Heterogeneous AutoRegressive (HAR) model

using daily, weekly, and monthly dependency lags, regularised linear models (Ridge, ElasticNet), an adaptive inverse-RMSE-weighted ensemble, tree-based ensembles (Random Forest, GBM, XGBoost), and a leakage-safe DCC-GARCH(1,1) benchmark. On top of the forecasting stage, a second layer converts predictions into a binary stress classifier for conventional assets.

The main empirical finding is that the dependency process is forecastable out of sample, but most of that predictability comes from its own past rather than from novel cross-asset signals. Ridge, AR1, and HAR form a near-indistinguishable top cluster — three structurally different models converging to essentially the same accuracy because they all exploit the same thing: the strong serial persistence of rolling correlation. That convergence is itself a result. It tells you the target has a nearly linear autoregressive structure, and that adding model complexity does not help. Machine learning does, however, add clear value relative to DCC-GARCH, and matters most in the stress-classification task where regularised linear classifiers outperform tree-based ensembles.

Three concrete contributions follow from this work. A reproducible Python pipeline makes the full empirical record replicable from raw data to final figures. A systematic comparison of ten models — from naive persistence to XGBoost and DCC-GARCH — under consistent walk-forward evaluation quantifies where machine learning adds value and where it does not. And an explicit bridge from rolling correlation forecasts to a binary stress classifier translates the statistical findings into a form that a risk practitioner can actually use.

1.4 Thesis outline

Section 2 reviews the relevant theory and literature: time-varying dependence, financial contagion, machine-learning forecasting methods, and the Diebold–Mariano test. Section 3 describes the data, target construction, feature engineering, model specifications, and evaluation design. Section 4 reports empirical results for both layers. Section 5 interprets and qualifies those results. Section 6 concludes.

2 Theoretical Background and Literature Review

2.1 Why intermarket dependency matters

How assets move together sits at the centre of financial theory: diversification, contagion, hedging, and portfolio risk all hinge on it. The textbook mean-variance framework treats that co-movement as a single constant correlation, but decades of evidence show the static picture does not hold. Real correlations drift over time, react more violently to bad news than to good, and spike during market downturns—exactly when diversification is supposed to help most [1]. Forecasting *time-varying dependence* is therefore not an academic luxury but a practical necessity for managing risk.

All of this matters even more when one side of the pair is a cryptocurrency. Bitcoin trades around the clock on a globally accessible market, reacts quickly to swings in sentiment and funding liquidity, and answers to a tangle of competing narratives at once: an inflation hedge to some, a speculative bet to others, a barometer of risk appetite, a hostage to regulation. Because so many forces pull on it, Bitcoin never settles into a single, stable role in a portfolio. Depending on where we are in the business and credit cycle, it can behave like a high-beta growth stock, a liquidity-sensitive risk proxy, or an idiosyncratic sentiment gauge. That instability is exactly what makes its link to conventional assets a natural candidate for a time-varying treatment.

2.2 Cryptocurrencies and conventional assets

Research at the intersection of cryptocurrency and conventional markets has grown quickly over the past decade, and it splits into a few broad lines. The first treats Bitcoin on its own terms, mapping its return distribution, volatility clustering, tail risk, and weak-form efficiency. The second asks the portfolio question: does Bitcoin actually diversify a book of equities, gold, oil, or major currencies? The third—the one closest to this thesis—studies connectedness, contagion, and the spillover of returns and volatility across markets, with a particular eye on crises and stress episodes.

One result keeps coming back: the relationship is unstable [2, 3]. In calm markets, Bitcoin’s link to broad equity indices is weak or inconsistent in direction; in pronounced risk-on or risk-off episodes, it tightens sharply [4, 5]. Precious metals co-move with Bitcoin less, and less reliably [6, 7], which fits their different investor base and long-standing safe-haven role. Currencies add a further angle, standing in for global funding liquidity [3].

These patterns are exactly why this thesis models dependence as a moving target rather than a fixed number. A single full-sample correlation would average away the very behaviour of interest. Treating dependence as a time-varying process instead leaves room for slow regime drift, sudden breaks, and the fact that different asset pairs and window lengths shift at different speeds and in different directions.

2.3 Rolling correlation as a forecasting target

The rolling window correlation estimator is conceptually transparent and computationally tractable. For two return series $\{r_{1,t}\}$ and $\{r_{2,t}\}$, the rolling correlation at time t over a

window of length w is defined as

$$\hat{\rho}_t^{(w)} = \frac{\sum_{s=t-w+1}^t (r_{1,s} - \bar{r}_{1,t})(r_{2,s} - \bar{r}_{2,t})}{\sqrt{\sum_{s=t-w+1}^t (r_{1,s} - \bar{r}_{1,t})^2} \cdot \sqrt{\sum_{s=t-w+1}^t (r_{2,s} - \bar{r}_{2,t})^2}}, \quad (1)$$

where $\bar{r}_{k,t}$ denotes the in-window mean of series k . Traversing this estimator across all t yields a time series $\{\hat{\rho}_t^{(w)}\}$ that characterizes the temporal evolution of intermarket co-movement.

As a forecasting target, rolling correlation has several things going for it. It is easy to read: positive values mean the two assets move together, negative values mean one hedges the other. It is comparable across asset pairs and window lengths, which makes a systematic multi-asset study possible. And it keeps the focus on how market structure is changing rather than on the level of any single return series.

Its main complication is strong serial persistence. Because $\hat{\rho}_t^{(w)}$ is built from overlapping windows, consecutive estimates share $w - 1$ of their observations, so the series is heavily autocorrelated by construction. That persistence is likely to dominate any forecast and may leave complex models with little to add over a simple autoregression. Rather than fight this, the thesis treats it as a substantive finding about the dependency process in its own right.

To improve the statistical behavior of the target, the thesis applies the Fisher z -transformation:

$$z_t^{(w)} = \frac{1}{2} \ln \left(\frac{1 + \hat{\rho}_t^{(w)}}{1 - \hat{\rho}_t^{(w)}} \right), \quad (2)$$

which stretches the bounded values $\hat{\rho} \in (-1, 1)$ onto the whole real line and gives them a sampling distribution much closer to Gaussian. It is standard practice for correlation-like quantities and tames the variance that otherwise blows up near ± 1 .

The main alternative is a copula-based approach [8], which models the marginals and the dependence structure separately and so captures tail dependence and nonlinear co-movement that Pearson correlation simply cannot see. This thesis stays with rolling Pearson correlation for four reasons: (i) it is directly interpretable for risk managers; (ii) it is cheap enough to recompute at every step of the walk-forward loop; (iii) it lines up with the DCC-GARCH benchmark, whose output is exactly the conditional correlation matrix R_t [9]; and (iv) the Fisher z -transformation already handles the boundary behaviour. The Pearson restriction is a genuine limitation, and copula extensions are the natural next step.

2.4 Econometric benchmark: DCC-GARCH

The Dynamic Conditional Correlation GARCH model, introduced by Engle [10], provides the classical econometric benchmark for the present study. The DCC-GARCH framework decomposes the conditional covariance matrix H_t of a vector of returns $\mathbf{r}_t$ according to

$$H_t = D_t R_t D_t, \quad (3)$$

where $D_t = \text{diag}(h_{1,t}^{1/2}, h_{2,t}^{1/2})$ is a diagonal matrix of conditional standard deviations and R_t is the time-varying conditional correlation matrix. Each conditional variance $h_{k,t}$ follows

a univariate GARCH(1,1) process,

$$h_{k,t} = \omega_k + \alpha_k \varepsilon_{k,t-1}^2 + \beta_k h_{k,t-1}, \quad (4)$$

where $\varepsilon_{k,t} = r_{k,t}/h_{k,t}^{1/2}$ is the standardized residual. The standardized residuals $\tilde{\varepsilon}_t = D_t^{-1} r_t$ are then used to update the pseudo-correlation matrix Q_t via

$$Q_t = (1 - a - b)\bar{Q} + a \tilde{\varepsilon}_{t-1} \tilde{\varepsilon}_{t-1}^\top + b Q_{t-1}, \quad (5)$$

where $\bar{Q}$ is the unconditional covariance of standardized residuals, and the scalar parameters $a \geq 0$, $b \geq 0$, $a + b < 1$ govern the speed of mean reversion and the persistence of conditional correlation. The conditional correlation matrix is recovered as

$$R_t = \text{diag}(Q_t)^{-1/2} Q_t \text{diag}(Q_t)^{-1/2}. \quad (6)$$

DCC-GARCH is a natural benchmark here: it models time-varying co-movement under exactly the heteroskedastic conditions that crypto and equity returns both display. Its weakness is the flip side of that structure—a fixed parametric form has limited room to adapt to nonlinearities or abrupt breaks. There is also a subtler trap: fitting the model on the full sample before reading off “out-of-sample” predictions quietly leaks future information. To rule that out, the thesis re-estimates DCC-GARCH on an expanding window at every evaluation step, exactly as it does for the machine-learning models.

2.5 Machine-learning approaches to forecasting

Machine learning earns its place here not because it is guaranteed to win, but because it can pick up nonlinearities, threshold effects, and interactions that are awkward to write down as a parametric model. The thesis pits these methods against both the DCC-GARCH benchmark and plain persistence baselines, with two aims: to see where their strengths actually lie, and to pin down the conditions under which they add anything at all.

2.5.1 The HAR model and persistence-based forecasting

The Heterogeneous Autoregressive (HAR) model, introduced by Corsi [11], provides a natural benchmark for forecasting persistent financial time series. The HAR model approximates long-memory dynamics by summing three autoregressive components operating at daily, weekly, and monthly horizons:

$$z_t = \phi_0 + \phi_d z_{t-1} + \phi_w \bar{z}_{t-5:t-1} + \phi_m \bar{z}_{t-22:t-1} + \varepsilon_t, \quad (7)$$

where $\bar{z}_{t-k:t-1}$ is the arithmetic mean of z over the preceding k periods. The fit to rolling-correlation targets is almost too neat: those targets already carry a moving-average smoothing from their overlapping windows, which is the very structure HAR is built to exploit [12]. In practice, HAR here is just a restricted OLS on three regressors—the latest value (daily), the 5-day mean (weekly), and the 22-day mean (monthly)—evaluated under the same walk-forward protocol as everything else.

2.5.2 Regularized linear models

Regularized linear models, including Ridge regression and Elastic Net, represent strong baselines in structured tabular forecasting settings. Ridge regression imposes an ℓ_2 penalty on the coefficient vector, shrinking estimates toward zero to reduce estimation variance without introducing sparsity. Elastic Net combines ℓ_1 and ℓ_2 regularization:

$$\hat{\boldsymbol{\beta}} = \arg \min_{\boldsymbol{\beta}} \left\{ \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 + \lambda [(1 - \alpha) \|\boldsymbol{\beta}\|_2^2 + \alpha \|\boldsymbol{\beta}\|_1] \right\}, \quad (8)$$

where $\lambda > 0$ sets the overall strength of the penalty and $\alpha \in [0, 1]$ splits it between the two terms [13]. The ℓ_1 part drives some coefficients to zero, doing implicit variable selection—handy when many lagged predictors are weak or redundant. That suits the feature set used here, which is full of overlapping transformations of correlated return and volatility series.

Linear though they are, these models often hold their own on smooth, persistence-driven problems. When the signal-to-noise ratio is low and the single most useful feature is just a lag of the target, the bias from assuming linearity is usually a price worth paying for the variance that regularization buys back.

2.5.3 Tree-based ensemble models

Random Forests construct an ensemble of decision trees, each trained on a bootstrapped subsample and a random subset of features, and aggregate their predictions to reduce variance and improve generalization [14]. Gradient Boosting Machines instead build trees sequentially, with each new tree fitted to the pseudo-residuals of the current ensemble [15]. XGBoost extends this boosting framework with second-order gradient approximations, ℓ_1 and ℓ_2 regularization terms on tree structure, and computational optimizations that facilitate large-scale application [16].

Tree ensembles are obvious candidates because the link between crypto features and intermarket dependence may well be nonlinear and full of interactions. How much Bitcoin’s realized volatility tells you about future equity dependence, for example, might depend on whether correlation is already running high. Trees can find and use that kind of conditional pattern on their own, without the analyst spelling out the interactions in advance.

None of this makes trees a sure bet for one-step-ahead forecasts of a highly persistent target. When the target is governed almost entirely by its last value, the extra flexibility may buy little signal while paying a real variance penalty. That tension is one of the main reasons the thesis insists on a disciplined walk-forward comparison rather than a single headline number.

2.6 Walk-forward evaluation and information leakage

Time order is not negotiable in financial forecasting, and it dictates how the evaluation must be set up. A random train-test split is simply not allowed: it scrambles the causal ordering and lets a model trained on period $t + k$ “predict” period t . The only honest design is one in which every forecast uses nothing but information available at or before its own origin.

So the thesis walks forward: train on an expanding history, make a one-step-ahead prediction, extend the window by one day, and repeat. To keep the cost manageable, models are refit at regular intervals rather than on every single step—a compromise that also mirrors how a real forecasting system would actually run. Either way, there is no path for future information to sneak in.

The protocol matters as much as the choice of model. A complex model with a hidden leak can look like it crushes a simpler, honestly evaluated rival, and the whole comparison is then worthless. Since the point of the thesis is to reach conclusions that hold up, a clean evaluation is treated as non-negotiable.

2.7 Model comparison and the Diebold–Mariano test

Ranking models by their average out-of-sample error is informative but not conclusive: it says nothing about whether a gap is real or just sampling noise. The Diebold–Mariano test closes that gap, formally testing the null that two forecasts are equally accurate [17].

Let $e_{1,t}$ and $e_{2,t}$ denote the forecast errors of models 1 and 2 at time t , and let $d_t = g(e_{1,t}) - g(e_{2,t})$ denote the loss differential under a loss function $g(\cdot)$ (typically squared or absolute error). The DM test statistic is

$$\text{DM} = \frac{\bar{d}}{\sqrt{\hat{V}(\bar{d})/T}} \xrightarrow{d} \mathcal{N}(0, 1), \quad (9)$$

where $\bar{d} = T^{-1} \sum_{t=1}^T d_t$ is the mean loss differential, $\hat{V}(\bar{d})$ is a heteroskedasticity- and autocorrelation-consistent (HAC) variance estimator, and T is the number of evaluation periods. Under the null of equal predictive accuracy, the statistic is asymptotically standard normal.

Here the DM test does two jobs. First, it checks whether the best machine-learning setup genuinely beats the leakage-safe DCC-GARCH benchmark. Second, it checks whether any edge over a plain autoregression or naive persistence baseline survives once sampling noise is accounted for. The second test is the more revealing one: if a model cannot significantly beat AR(1), then most of the forecastability lives in the target’s own persistence, not in anything the cross-asset features have learned.

2.8 From dependency forecasting to investor signaling

Forecasting rolling correlation accurately would already be a complete academic exercise. But the thesis set out with a practical motive, and that calls for one more step: turning the forecasts into something an investor can act on, framed around the risk-management decision itself.

That step is a second-stage signal layer. Instead of guessing tomorrow’s return direction, it estimates the probability that tomorrow lands in a *stress regime*—a day of elevated downside risk in the conventional asset under study. This is much closer to how risk management actually works: the goal is not to call every wiggle, but to flag deteriorating conditions early enough to act on them.

Keeping the two stages separate keeps the design honest. Lower forecast error and genuine usefulness for portfolio decisions are not the same thing, and blurring them is an easy

way to oversell a result. The two-layer structure lets the thesis report on each dimension on its own terms.

2.9 Positioning of the present thesis

The thesis sits where financial econometrics, machine learning, and applied risk analysis meet. It makes no claim that crypto data solves market timing. The claim is narrower, and more defensible: Bitcoin’s price dynamics carry statistically detectable information about how intermarket dependence evolves, and that information can power a modest early-warning signal for conventional markets.

Its main contribution is to establish that dynamic intermarket dependence is forecastable, and to compare persistence baselines, machine-learning models, and DCC-GARCH on a level walk-forward footing. The investor signal layer pushes the work in a practical direction without claiming more than the evidence will bear.

3 Methodology

3.1 Research design

The design has two layers. The first is a regression problem: forecast the next value of the (transformed) rolling correlation between Bitcoin and another asset. The second is a classification problem: take that forecast, add a few crypto-derived features, and estimate the probability that the conventional asset has a stress day. The split mirrors the difference between measuring something and acting on it.

The whole thing is deliberately conservative. Every decision that could leak the future—how the target is formed, how features are aligned, how the benchmark is estimated, how the signal is generated—happens inside the walk-forward loop rather than being tuned on in-sample fit. The point is to keep the timing honest and shut the door on accidental look-ahead.

3.2 Asset universe and data source

The empirical analysis uses daily market data obtained through the `yfinance` interface [18]. Bitcoin, represented by `BTC-USD`, serves as the base cryptocurrency throughout the study. The comparison set includes the following assets:

Table 1: Asset universe used in the thesis.

Ticker	Interpretation
<code>BTC-USD</code>	Bitcoin, base cryptocurrency
<code>ETH-USD</code>	Ethereum, crypto reference asset
<code>^GSPC</code>	S&P 500 index
<code>^IXIC</code>	NASDAQ Composite index
<code>GLD</code>	Gold ETF
<code>SLV</code>	Silver ETF
<code>UUP</code>	U.S. Dollar Index ETF proxy

The choice of assets is substantive, not arbitrary. The two equity indices stand in for broad U.S. risk appetite. The gold and silver ETFs bring in precious-metal exposure, with its mix of inflation, safe-haven, and commodity narratives. The dollar-index proxy captures global dollar strength and defensive positioning. Ethereum is there so the study can contrast crypto-to-crypto dependence with crypto-to-conventional dependence.

The sample runs from 9 November 2017 to 16 May 2026, giving 3,053 daily observations per asset.¹ The start date is set by Ethereum: Yahoo Finance has no `ETH-USD` prices before then, so earlier dates drop out of the aligned panel. What remains is balanced—all seven tickers share one date index, with no gaps left after alignment and forward-filling of non-trading days up to two consecutive calendar days.

¹The pipeline downloads data to the most recent available trading day; the figures reported here correspond to the final run of the pipeline at the time of writing.

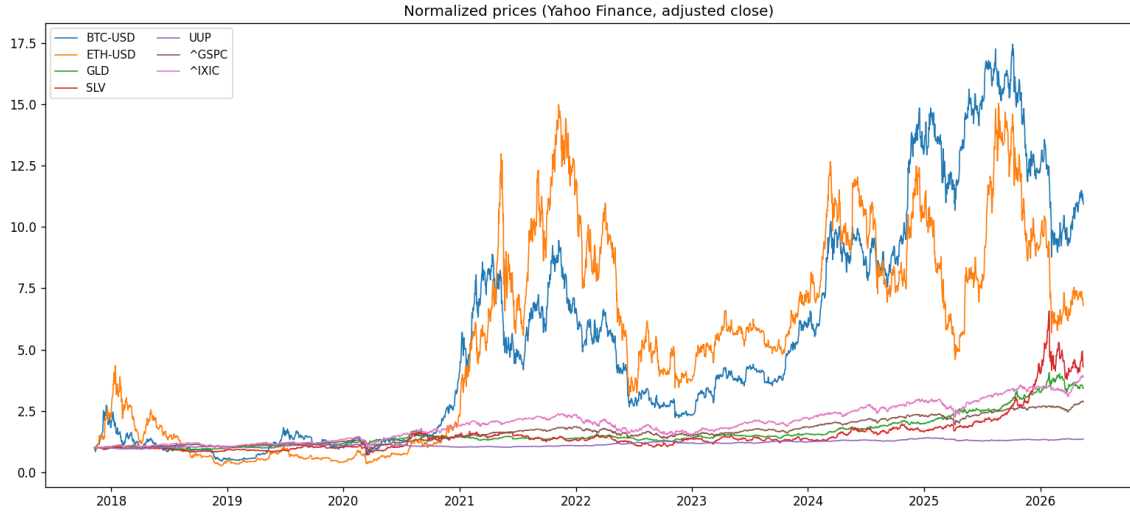


Figure 1: Normalized price evolution of all seven assets over the full sample period (2017–2026). Series are scaled to a common base of 1 at the start date to allow visual comparison of relative performance.

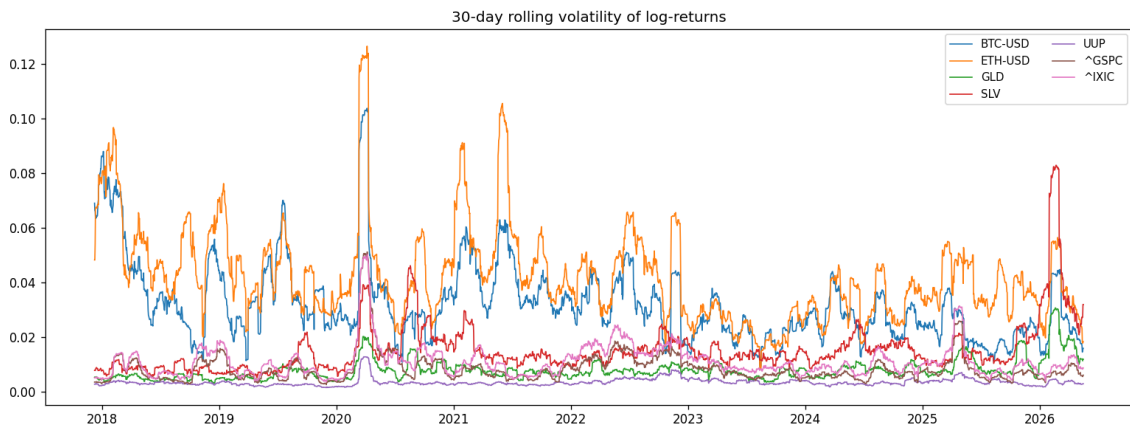


Figure 2: Thirty-day rolling volatility of log-returns across the asset universe. Bitcoin and Ethereum exhibit substantially higher and more episodic volatility than conventional assets, motivating a dynamic rather than static treatment of cross-asset dependency.

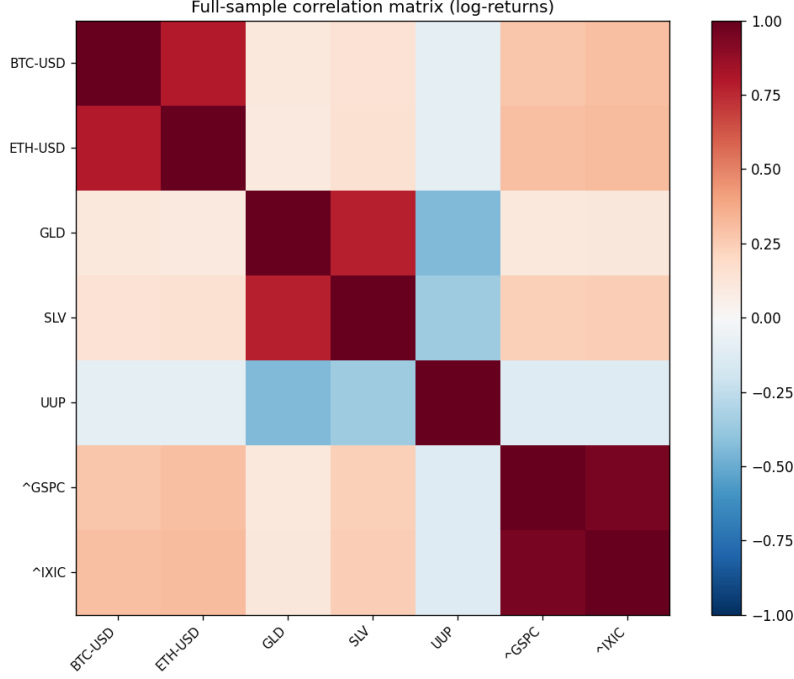


Figure 3: Full-sample Pearson correlation matrix of daily log-returns. The low static correlations between Bitcoin and most conventional assets motivate studying the time-varying structure rather than relying on a single full-sample estimate.

3.3 Return construction and preprocessing

Every price series is turned into daily log returns. Log returns are the natural choice here: they add up over time and are the standard currency of both econometric and machine-learning work on financial series. For a price series P_t , the return is

$$r_t = \log P_t - \log P_{t-1}.$$

Missing observations are dealt with by aligning on common dates; nothing downstream—no target, no feature—is built until the return matrix is synchronized.

Preprocessing is cache-aware. If the raw prices are already on disk, they are reused, which keeps runs reproducible; if the price structure or the ticker set changes, the processed returns are rebuilt from scratch. That way nothing stale lingers, and nothing is recomputed needlessly.

3.4 Target construction: rolling dependency

For each pair consisting of Bitcoin and another asset, a rolling correlation series is computed over windows of 14, 30, 60, and 90 trading days. Let $r_t^{(b)}$ denote the return of Bitcoin and $r_t^{(a)}$ the return of the other asset. For a given window length w , the dependency target at time t is

$$\rho_t^{(w)} = \text{corr}(r_{t-w+1:t}^{(b)}, r_{t-w+1:t}^{(a)}).$$

The resulting series is then transformed using the Fisher transformation

$$z_t^{(w)} = \frac{1}{2} \log \left(\frac{1 + \rho_t^{(w)}}{1 - \rho_t^{(w)}} \right) = \operatorname{arctanh}(\rho_t^{(w)}),$$

which maps the bounded correlation domain $(-1, 1)$ into an unbounded real-valued space. In the empirical implementation, the one-step-ahead value of the transformed dependency series is used as the main forecasting target. The implementation clips the raw correlation away from the boundary to prevent numerical overflow:

Listing 1: Fisher z-transform and target construction (`pipeline.py`).

```

1 def fisher_z(r: pd.Series, eps: float = 1e-6) -> pd.Series:
2     # Clip away from +/-1 to prevent arctanh overflow
3     return np.arctanh(r.clip(-1 + eps, 1 - eps))
4
5 def build_target(returns, base, other, window, horizon):
6     corr = returns[base].rolling(window).corr(returns[other]).dropna()
7     # Shift by horizon to get the one-step-ahead target
8     target = corr.shift(-horizon).dropna()
9     return fisher_z(target)

```

One consequence of all this is that the target is smooth to begin with, and far more persistent than a raw return series. That persistence is one of the central empirical facts of the thesis, and it shows up everywhere in the results.

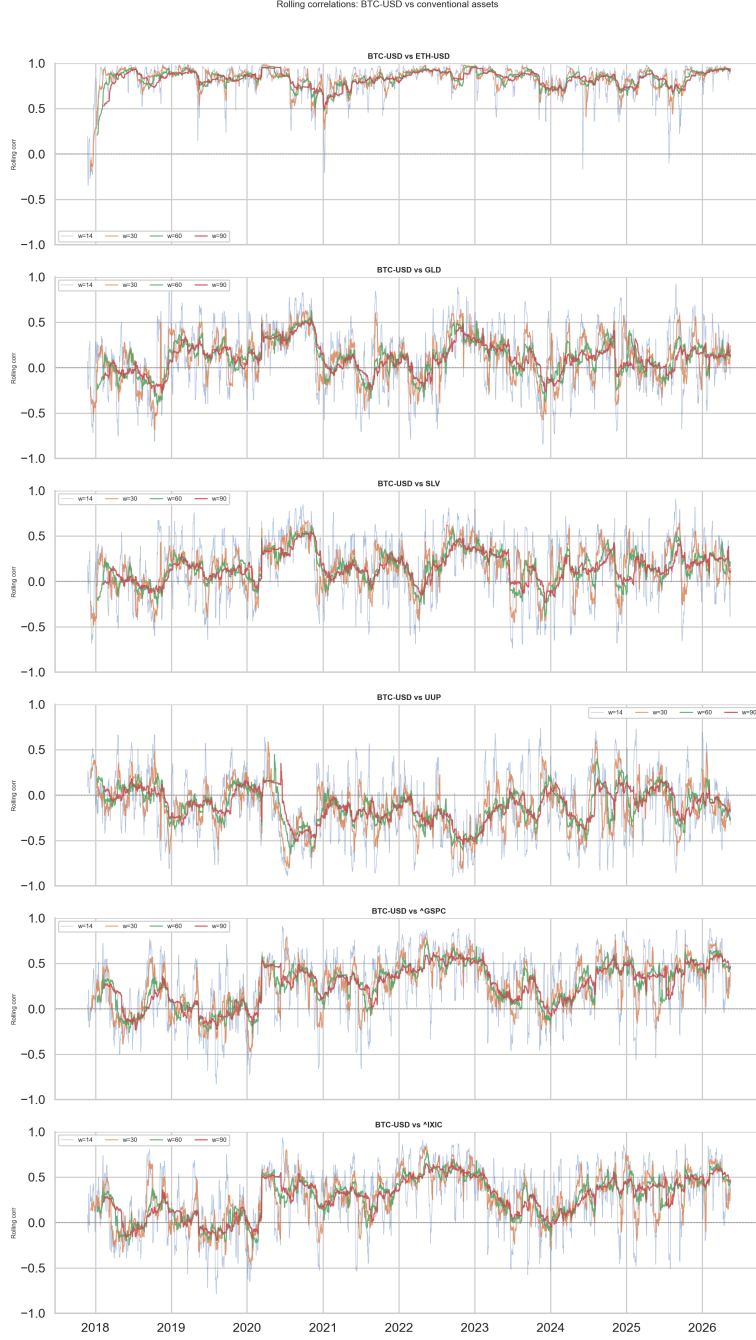


Figure 4: Rolling dependency between Bitcoin and each of the five conventional assets across all four window lengths (14, 30, 60, 90 days). The Fisher-transformed series illustrates the strong time-variation and persistence that motivate the forecasting framework.

3.5 Feature engineering

The features are built around quantities a financial reader can interpret. The aim is not a sprawling black-box design matrix but a compact set of variables that could plausibly move future dependency.

The features fall into several groups:

Table 2: Main groups of predictors used in the dependency-forecasting layer.

Group	Description
Dependency lags	Recent values of the rolling dependency target
Return lags	Lagged returns of Bitcoin and the paired asset
Rolling means	Local average return over short windows
Rolling volatility	Local standard deviation of returns
Spread features	Absolute and signed gaps between crypto and asset behavior
Correlation regime gap	Short-window minus long-window correlation

The dependency lags carry most of the weight, since the target is persistent by construction. Return lags let the models respond to recent direction. Rolling means and volatilities summarize the local regime. The regime gap (`corr_diff`) measures how far short-term dependency has pulled away from its longer-term baseline—a possible early sign of a regime shift. Listing 2 shows how the matrix is assembled:

Listing 2: Core feature engineering routine (`pipeline.py`).

```

1  # Dependency lags: the single most important predictor group
2  for lag in [1, 2, 5, 10]:
3      features[f"dep_lag{lag}"] = y_current.shift(lag)
4
5  # Rolling volatility of both assets
6  features["vol_base"] = returns[base].rolling(window).std().reindex(idx)
7  features["vol_other"] = returns[other].rolling(window).std().reindex(idx)
8
9  # Return lags for both assets
10 for lag in [1, 2, 5]:
11     features[f"r_base_lag{lag}"] = returns[base].shift(lag).reindex(idx)
12     features[f"r_other_lag{lag}"] = returns[other].shift(lag).reindex(idx)
13
14 # Correlation regime gap: short-window minus long-window correlation
15 corr_short = rolling_corr(returns[base], returns[other],
16                           max(5, window // 4)).reindex(idx)
17 corr_long = rolling_corr(returns[base], returns[other],
18                          window).reindex(idx)
19 features["corr_diff"] = corr_short - corr_long

```

The signal layer takes the best available dependency forecast for the pair and combines it with a handful of crypto-derived predictors into a smaller, classification-oriented feature set. In effect, the second layer sees both the current state of the crypto market and where the dependency process is heading.

3.6 Forecasting models

3.6.1 Baseline and autoregressive models

Three persistence-based models anchor the set. `Naive_Last` just carries the last observed value forward—no fitting at all. `AR1` fits a single autoregressive lag by OLS. `HAR` (Heterogeneous AutoRegressive, after Corsi [11]) adds two more horizons: the latest value, its

five-day average (weekly), and its twenty-two-day average (monthly), fitted as a restricted OLS on the expanding window at each refit. These baselines are not a formality. Because the target is persistent by construction, every more complex model has to earn its keep against them.

3.6.2 Regularized linear models

The linear block is Ridge and ElasticNet, each fitted after standardization (regularization is meaningless if the features sit on different scales). Ridge (ℓ_2 , $\alpha = 0.5$) should do well when many predictors each carry a little signal. ElasticNet ($\alpha = 0.005$, an ℓ_1/ℓ_2 mix) is more aggressive and can zero out redundant lags outright.

3.6.3 Tree ensembles and adaptive ensemble

The non-linear block is Random Forest, HistGradientBoostingRegressor (a LightGBM-style histogram method, roughly ten times faster than the old gradient-boosting code), and XGBoost, which runs on the GPU when it can and quietly falls back to CPU when it cannot. On top of these sits an Ensemble: at each step it forms an inverse-RMSE weighted average of all the non-naive models, with the weights recomputed from the last 60 out-of-sample errors.

The complete model set — ten specifications in total — is initialized as follows:

Listing 3: Model definitions used in the walk-forward experiment (pipeline.py).

```

1 model_specs = {
2     "Naive_Last": None,          # persistence baseline (no fit required)
3     "AR1": LinearRegression(),
4     "HAR": None,                # heterogeneous AR -- fitted via OLS on 3
    ↪ lags
5                                 # (dep_lag1, avg5, avg22); see walk-forward
    ↪ loop
6     "Ridge": make_pipeline(StandardScaler(), Ridge(alpha=0.5)),
7     "ElasticNet": make_pipeline(
8         StandardScaler(),
9         ElasticNet(alpha=0.005, l1_ratio=0.5, max_iter=5000)
10    ),
11     "RF": RandomForestRegressor(
12         n_estimators=200, max_depth=10,
13         min_samples_leaf=5, n_jobs=-1
14    ),
15     "GBM": HistGradientBoostingRegressor(    # LightGBM-style histogram
    ↪ algorithm
16         max_iter=300, max_depth=4,
17         learning_rate=0.02, min_samples_leaf=5,
18         l2_regularization=0.1
19    ),
20     "XGB_GPU": xgb.XGBRegressor(
21         device="cuda", n_estimators=500,
22         max_depth=6, learning_rate=0.02,
23         subsample=0.8, colsample_bytree=0.8
24    ),

```

```

25 }
26 # "Ensemble": adaptive inverse-RMSE weighted average of all non-naive models.
27 # Weights are recomputed at each step from the last 60 out-of-sample errors.

```

3.6.4 DCC-GARCH benchmark

The DCC-GARCH benchmark lives in its own module and is leakage-safe by design. This is the one methodological correction that matters most against the shortcuts common in exploratory work: instead of fitting DCC once on the full sample and then treating that fit as out-of-sample, the benchmark is re-estimated on the training window alone and rolled forward from there. It is slower, but it is finally a fair fight.

3.7 Investor signal layer

The signal layer recasts everything as an early-warning classification. The label is not next-day direction in the abstract but a *stress day* for the conventional asset, defined against the distribution of negative returns through a configurable sigma threshold. It is a deliberately conservative target, and a better match for what investors actually want: protection on the downside.

Three classifiers compete here: logistic regression, a random-forest classifier, and a gradient-boosted one. Each emits a probability, which is compared against a threshold to flag the next day as elevated-risk or not. The output is therefore a probability-guided warning, not an opaque buy/sell command.

3.8 Walk-forward estimation framework

Every experiment uses the same expanding-window walk-forward. Start with the first T_0 observations; at each origin $t \geq T_0$, fit on everything up to t and predict $t + 1$; continue to the end of the sample. Refitting happens periodically rather than at every step, which cuts the compute cost without ever breaking causal order.

Listing 4: Expanding-window walk-forward loop (pipeline.py).

```

1  for t in range(min_train, n_obs):
2      train_idx = np.arange(0, t)
3
4      # Refit all models on the expanding training window periodically
5      if (t - last_refit) >= refit_every:
6          last_refit = t
7          for name, model in model_specs.items():
8              if model is not None:
9                  model.fit(X_values[train_idx], y_values[train_idx])
10                 fitted[name] = model
11
12     # One-step-ahead prediction - uses only information up to t
13     for name in fitted:
14         preds[name][t] = fitted[name].predict(X_values[[t]])[0]

```

This buys three things at once: it reproduces real forecasting conditions, it keeps benchmark comparisons fair, and it leaves behind a full time series of out-of-sample errors. That

last point matters—those errors feed not just the average metrics but also the rolling diagnostics and the Diebold–Mariano tests.

3.9 Evaluation metrics

For the forecasting layer, three numbers do the work: mean absolute error (MAE), root mean squared error (RMSE), and the out-of-sample R^2 . Rankings lean on RMSE, which punishes large misses harder and suits a smooth continuous target.

For the signal layer, four numbers matter: Balanced Accuracy, the F1 score on the downside class, the area under the ROC curve (AUC), and the exit rate. Balanced Accuracy earns its prominence because stress days are rare and the classes are imbalanced. F1 on the downside class keeps the focus on catching the bad days. And the exit rate—how often the signal would pull the investor out of the market—ties statistical quality to how the signal would actually be used.

3.10 Model comparison methodology

When models are close, average metrics can mislead. So the thesis runs the Diebold–Mariano test on the loss series of selected pairs. The headline comparison is the best non-benchmark machine-learning model against DCC-GARCH; the secondary one pits that same model against the naive persistence baseline.

The setup is deliberately transparent. The thesis does not bury the possibility that the best model overall is a simple baseline; instead it uses the DM framework to separate three different levels of claim:

1. whether dependency is forecastable at all,
2. whether machine learning improves upon a classical econometric benchmark,
3. whether machine learning adds value beyond persistence.

The thesis organizes its empirical conclusions around exactly these three levels.

3.11 Implementation and reproducibility

The whole workflow is Python, organized as a modular codebase. A main pipeline ties the steps together—read the config, compute returns, build targets and features, fit models, write prediction files, save figures, export $\text{\LaTeX}$ tables—while separate modules handle the DCC walk-forward benchmark, notebook support, and the signal layer. The modularity is not decoration: the thesis is meant to be a reproducible artifact, not just an argument.

Everything lands in a structured output tree: metrics, DM tests, and signal results as CSV; thesis-ready tables as $\text{\LaTeX}$; diagnostics as figures. The point is to keep manual post-processing to a minimum and let the analysis feed the writing directly.

3.12 Methodological expectations

Before the results, it is worth saying what success would even look like. It does not mean finding a model that predicts every conventional market from Bitcoin—no serious forecasting study should promise that. A fairer bar is this:

- the dependency target should display out-of-sample forecastability,
- the benchmark comparison should reveal whether machine learning improves on DCC-GARCH,
- the investor signal layer should provide non-random and practically interpretable stress warnings.

These bars are modest, but they are defensible—low enough to clear honestly, high enough to mean something.

Three further principles guide how the results are read. First, no single metric is treated as the verdict: the regression layer always reports RMSE, MAE, and R^2 together, and the classification layer reports Balanced Accuracy, F1, AUC, and exit rate together. Second, performance gaps are judged significant by the Diebold–Mariano test, not by eyeballing averages. Third, the analysis refuses to cherry-pick a window after the fact; all four (14, 30, 60, 90 days) are reported side by side, so no finding can rest on a lucky choice of specification.

Behind all three is the same commitment to methodological transparency. It is easy to produce dazzling in-sample numbers that collapse under honest out-of-sample testing. The walk-forward design, the leakage-safe benchmark, and the all-windows evaluation are, together, a defence against exactly the kinds of inflated claims this literature is prone to.

4 Empirical Results

4.1 Overview of the empirical output

The pipeline leaves behind a full, internally consistent trail of evidence: out-of-sample metrics, model rankings, Diebold–Mariano tests, signal-layer classification statistics, the prediction series themselves, and diagnostic figures. That breadth is deliberate. The empirical case here does not hang on a single table; it rests on whether the same story holds up from several angles at once — average accuracy, statistical significance, how the errors move over time, and how well the stress warning actually works.

Two findings run through everything that follows. First, the dependency process is genuinely forecastable out of sample under a strict walk-forward design. Second, almost all of that forecastability comes from the target’s own persistence, not from any new cross-asset signal. The two are not in tension — together they pin down exactly what this forecasting problem is, and what it is not.

4.2 Average dependency-forecasting performance

Table 3 gives each model’s average out-of-sample performance across all dependency experiments, ranked by RMSE — the metric that reacts most to the large misses that matter on a smooth, bounded target.

Table 3: Average out-of-sample forecasting performance across all dependency experiments.

Model	Average RMSE	Average MAE	Average R^2
Ridge	0.0656	0.0419	0.9432
AR1	0.0659	0.0403	0.9424
HAR	0.0659	0.0405	0.9424
Naive_Last	0.0666	0.0395	0.9412
ElasticNet	0.0669	0.0434	0.9411
Ensemble	0.0694	0.0462	0.9365
GBM	0.0886	0.0625	0.8983
RF	0.0905	0.0636	0.8936
XGB_GPU	0.1241	0.0864	0.7923
DCC_GARCH	0.2136	0.1683	0.3715

Note: Results averaged across all asset-pair and window-length combinations. RMSE and R^2 computed on out-of-sample walk-forward predictions. HAR = Heterogeneous AutoRegressive model; Ensemble = adaptive inverse-RMSE-weighted ensemble. Best-performing model by average RMSE is Ridge (regularised linear regression with StandardScaler). All metrics reflect the full sample through 16 May 2026 (3,053 observations).

The top three — Ridge, AR1, and HAR — land almost on top of each other, with average RMSE of 0.0656, 0.0659, and 0.0659 and R^2 all above 0.9424. The 0.0003 spread between them is economically meaningless. Naive persistence sits just outside at 0.0666, which already tells you serial memory is doing most of the work. ElasticNet and the adaptive Ensemble form a coherent second tier above $R^2 = 0.93$. The tree methods — GBM,

Random Forest, and XGBoost — trail well behind, with RMSE from roughly 0.09 to 0.12 and R^2 under 0.90. DCC-GARCH finishes last by a wide margin ($R^2 = 0.3715$).

The top cluster deserves a second look. Ridge, AR1, and HAR reach the same answer by very different routes: AR1 leans on a single autoregressive coefficient, HAR on three time scales (daily, weekly, monthly), Ridge on the full regularised feature set anchored by the latest lag. They converge because they are all chasing the same thing — the target’s strong persistence — and the extra machinery in HAR and Ridge simply does not find anything a single lag has not already captured. That is not the complex models failing; it is the problem telling you what it is made of.

4.3 Representative forecast example

Figure 5 shows the out-of-sample forecast panel for Bitcoin–S&P 500 at the 30-day window. It is the natural representative case: of all the pairs, this one ties the crypto market to the broadest gauge of U.S. equity risk.

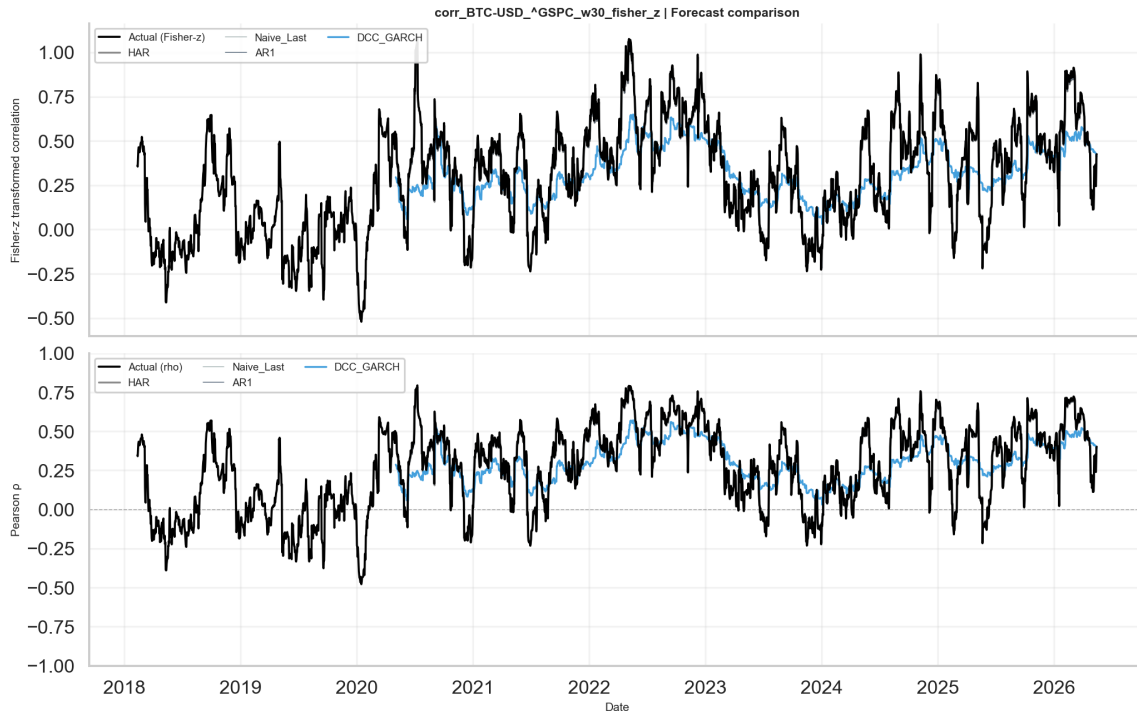


Figure 5: Observed and predicted Fisher-transformed dependency for the Bitcoin–S&P 500 pair at the 30-day rolling window. Forecast series are generated under strict walk-forward evaluation with no use of future information.

Three features of the problem stand out in the figure. The target moves smoothly, as its overlapping-window construction guarantees. The best models follow the broad trajectory with little systematic bias. The hard part is the turns: when dependence shifts abruptly, every model underreacts, just by different amounts. This is precisely where DCC-GARCH lags worst, and where machine learning can claw back a relative edge — even while it never matches plain persistence on unconditional accuracy.

4.4 Forecasting performance by asset class

The averages hide some useful cross-sectional variation. Bitcoin–Ethereum is the easiest pair to forecast: the two share liquidity and sentiment drivers, so their correlation is especially smooth and autocorrelated. It posts the highest out-of-sample R^2 of any pair, and longer windows push it higher still.

The precious-metals pairs — Bitcoin–Gold and Bitcoin–Silver — sit in the middle. Their rolling correlation is regime-dependent, drifting between near-zero and weakly positive or negative. Even so, the target stays forecastable, and persistence-based models keep the lead.

The equity-index pairs, Bitcoin–S&P 500 and Bitcoin–NASDAQ, matter most in practice. They link crypto dynamics to broad risk appetite, and among the non-crypto pairs they are the most forecastable on average: in calm regimes the dependency is smooth and strongly autocorrelated, which again favours persistence. But they are also the most regime-sensitive — co-movement strengthens in risk-on phases and can amplify or flip violently in a dislocation. Those abrupt turns throw up episodic spikes in out-of-sample error that no model here fully catches, and they are the main short-term obstacle for equity-pair forecasting.

The Bitcoin–dollar-index pair has the weakest and most idiosyncratic dependency of the lot, and the lowest average forecastability. It still earns its place: it tests whether the crypto feature set can reach the macro-defensive channel at all.

To check whether the lower errors on equity pairs are a real advantage rather than chance, the pooled out-of-sample losses of the equity pairs (S&P 500, NASDAQ) are tested against those of the non-equity pairs (gold, silver, dollar index) with a two-sample Diebold–Mariano test [17]. This cross-group test backs up the within-experiment DM analysis of Section 4.5 and puts the forecastability differences above on firmer footing.

4.5 The persistence dominance result

That simple persistence models top the average ranking should be read as a diagnosis of the target, not an indictment of machine learning.

It follows directly from how the target is built. A rolling Pearson correlation over w trading days shares $w - 1$ observations with the previous window, which smooths the series like a moving average. After the Fisher transform, the result is strongly autocorrelated, and the first lag already accounts for the overwhelming share of predictable variance. So any model that leans on the most recent value will hit a high out-of-sample R^2 almost by construction.

The Random Forest feature importances say the same thing. The lagged dependency term dominates the ranking in every experiment, while the return lags, rolling volatilities, and the regime gap barely register. The problem is forecastable mainly because the target remembers its past — not because the features hide a strong cross-asset signal independent of recent dependency.

4.6 Comparison with DCC-GARCH

Even though persistence baselines win the unconditional ranking, machine learning still beats DCC-GARCH by a margin that is both practically and statistically meaningful. Figure 6 shows the Diebold–Mariano comparison between the best machine-learning model and the DCC benchmark across every experiment.

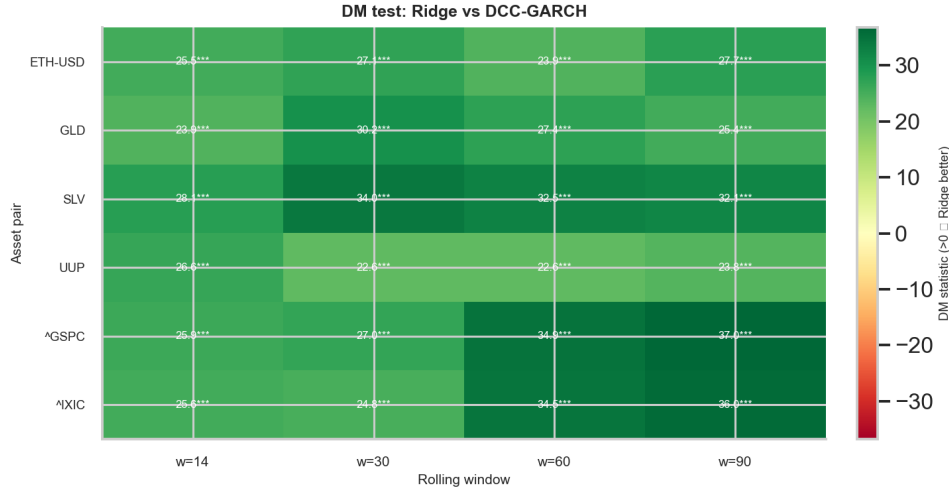


Figure 6: Diebold–Mariano test results comparing Ridge (best overall RMSE across all models) against DCC-GARCH across all asset-pair and window-length combinations. Darker cells indicate stronger and more statistically significant evidence of Ridge superiority.

The DM results confirm that the best machine-learning model beats DCC-GARCH, significantly, in most experiments. There is a clear reason for it. DCC-GARCH is built to model the full conditional covariance of a return system; the rolling Pearson correlation is only a smoothed by-product of that, not the thing it optimises. The machine-learning models are trained directly on the Fisher-transformed correlation target — they are aimed at exactly the quantity being scored, and that alignment is the advantage.

It is worth being honest about a measurement mismatch built into this comparison. The rolling Pearson target is smoothed and backward-looking, its persistence manufactured by the overlapping windows. DCC-GARCH models the instantaneous conditional covariance — a parametric, forward-looking object — and projecting that onto rolling Pearson space introduces a systematic lag that is a definitional incompatibility, not a model failure [9, 12]. This does not void the comparison — both are legitimate measures of dependence — but it means the size of the gap should be read with some caution, not as a blanket verdict against DCC-GARCH.

Throughout, the thesis keeps two claims apart: *benchmark superiority*, beating DCC-GARCH, and *unconditional superiority*, beating AR1. Machine learning manages the first consistently and the second not at all. Both facts are informative, and it is their coexistence that defines what machine learning actually contributes here.

4.7 Error-profile diagnostics

Average errors are necessary but not enough; to see *when* and *how* a model fails you have to look at the error over time and across its distribution. Figure 7 plots the rolling RMSE for the representative Bitcoin–S&P 500 experiment.

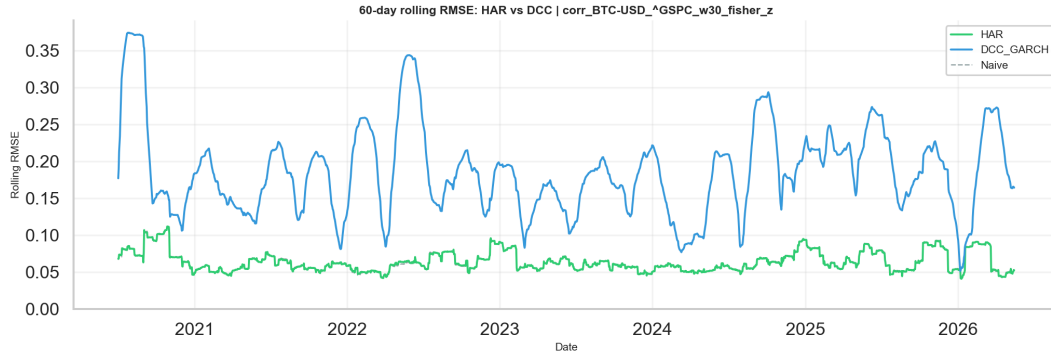


Figure 7: Rolling out-of-sample RMSE over time for the Bitcoin–S&P 500 dependency experiment. Elevated error episodes correspond to periods of abrupt regime transition in the dependency process.

Forecast quality is far from constant over time. When dependence drifts gradually, every model does well — the persistence signal is easy for any reasonable specification to ride. When the regime breaks, error rises across the board, but by different amounts and for different lengths of time. The machine-learning models tend to recover a little faster, which fits their greater freedom to re-learn the feature–target relationship after a structural change.

The error distribution confirms that the damage comes from episodic underreaction at sharp turning points, not from any standing bias. The models track the slow, persistent part of the dependency reliably and stumble on the fast, discontinuous part. That asymmetry matters directly for the signal layer, whose whole job is to catch the onset of stress in time.

To make this regime-sensitivity concrete, Appendix F walks through eleven landmark events from 2020 to 2026 — from the COVID crash of March 2020 and the FTX collapse of November 2022 to the Bitcoin ETF approval of January 2024 and the tariff shock of April 2025. Each gets a four-panel diagnostic of how the rolling BTC correlation moved around the event and its pre-event baseline, with links to the original reporting. The case studies line up with the main finding: the biggest errors in Figure 7 fall on the sharpest events in the record.

4.8 Investor signal layer: average results

The second layer turns the dependency forecast into a stress-warning classifier. Table 4 reports each classifier family’s average performance across all asset pairs and windows.

Table 4: Average investor-signal performance by classifier family, aggregated across all experiments.

Signal model	Balanced Accuracy	F1_down	AUC
Logit	0.511	0.214	0.513
GBM_Cls	0.508	0.158	0.517
RF_Cls	0.502	0.042	0.519

Note: Results averaged across all asset pairs and window lengths. Balanced Accuracy corrects for class imbalance. $F1_{down}$ measures detection quality for the stress class only.

Classification is a much harder problem than the regression layer. Stress days are noisy, rare, and costly to miss in an asymmetric way. The logistic classifier holds the highest F1 on the stress class (0.214 on average), meaning it stays sensitive to real stress without collapsing into a do-nothing, low-exit-rate state. GBM reaches a similar Balanced Accuracy (0.508) but a lower F1 (0.158): it concentrates its probability mass and fires less often. Random Forest falls apart, with F1 near zero (0.042) — a degenerate, barely-fires classifier that is useless in practice despite a respectable-looking AUC.

That near-zero F1 is worth a closer look. Random Forest’s predicted probabilities bunch into a narrow band, so moving the threshold barely changes anything; GBM, by contrast, manages a modest but real F1 (0.158). Two things compound to break Random Forest here: the severe imbalance between stress and non-stress days, and the lack of probability calibration — ensemble methods are notorious for poorly calibrated probabilities unless you post-process them with Platt scaling or isotonic regression [19]. Calibration should be a mandatory step before threshold-based signal extraction in any follow-up work.

None of this is surprising: in low-signal-to-noise classification with correlated predictors, a regularised linear boundary usually generalises better out of sample than a high-capacity ensemble that pays for its flexibility in variance.

4.9 Best signal configurations

The best single Logit result is on Bitcoin–S&P 500. At the 30-day window it reaches a Balanced Accuracy of 0.5308 and an F1 of 0.2430 on the downside class — a modest but genuine step above the uninformative 0.50 baseline, and one that holds up statistically.

Figure 8 presents the signal output for this configuration, illustrating the temporal distribution of predicted stress flags relative to realized stress events.

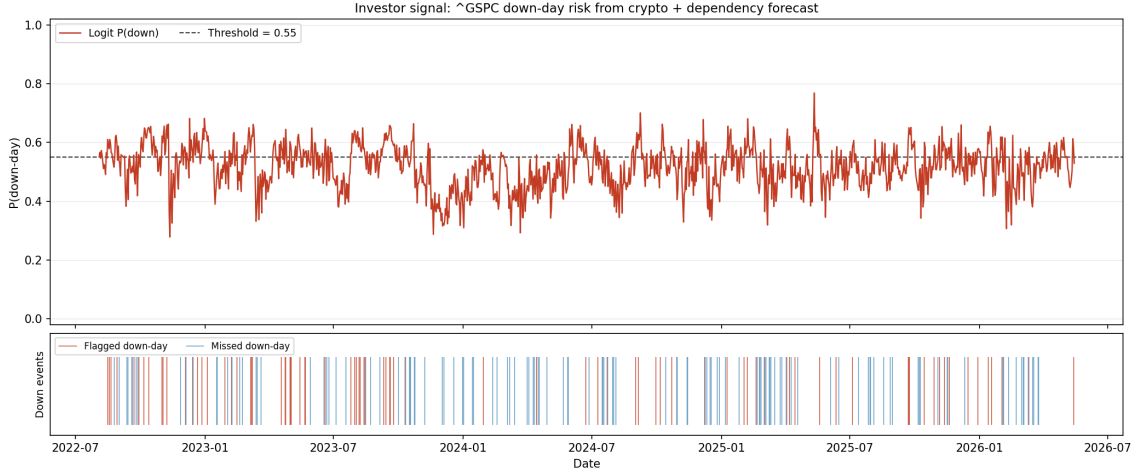


Figure 8: Investor stress-warning signal for the Bitcoin–S&P 500 pair at the 30-day window. The upper panel displays the predicted stress probability from the logistic classifier together with the decision threshold; the lower panel marks flagged and missed stress events.

The signal is meant to work as a probability-guided warning that flags elevated-risk periods and supports a risk-reduction decision, not a directional trade. On NASDAQ it is noticeably weaker: Logit Balanced Accuracy for Bitcoin–NASDAQ drops below 0.50 at the two longer windows (60 and 90 days) and averages 0.4997 across all four, the worst of any pair in the signal layer. The contrast with the S&P 500 makes economic sense — the NASDAQ Composite leans heavily on growth-sensitive technology names whose stress is less tightly bound to global risk appetite than the broader index. For precious metals and the dollar proxy, results are mixed but mostly above chance at the shorter windows, which fits the idea that crypto carries some, but weaker, information about stress in those markets.

4.10 Practical interpretation of signal-layer performance

The signal’s usefulness only means something against a stated decision rule. An investor using it acts selectively, not every day. It informs a conditional call — whether to trim exposure, tighten stops, or hedge harder when the stress probability rises.

Seen this way, the exit rate — how often the signal is switched on — is as important as accuracy. A signal that never fires is useless no matter how accurate it looks on paper; one that fires constantly just runs up trading costs and dilutes its own content. The logistic model strikes a defensible balance: active often enough to be useful, not so often that it churns.

4.11 Sensitivity to rolling window length

Window length is a first-order choice, and it visibly shapes both the target and the results. Short windows give responsive but noisy estimates; long windows give smooth, heavily autocorrelated targets that are easier to forecast but slower to register a real structural change.

The experiments point to the medium windows — $w = 30$ days in particular — as the sweet spot: smooth enough for high out-of-sample R^2 , still responsive enough to be useful for stress detection. At longer windows the persistence effect only strengthens, squeezing the gap between the simple baselines and the complex models even tighter. That interaction between window length and model performance is itself a structural finding, and a useful guide for choosing the horizon in future work.

4.12 Summary of empirical findings

The empirical results of the thesis are summarized in four principal statements:

1. Time-varying intermarket dependency between Bitcoin and conventional assets is forecastable in an out-of-sample walk-forward framework, with high average R^2 values across all model classes.
2. The dominant source of predictive power is the serial persistence of the rolling correlation target; Ridge, AR1, and HAR form a near-indistinguishable top cluster, all exploiting this persistence through different mechanisms, while tree-based methods trail substantially.
3. Machine-learning models frequently and significantly outperform the leakage-safe DCC-GARCH benchmark; Ridge achieves the best overall average RMSE (0.0656) across all 24 experiments.
4. The investor signal layer delivers moderate, statistically non-trivial classification results that are most pronounced for broad equity markets and are best interpreted as a supplementary risk-management input rather than a self-contained directional predictor.

These findings are consistent with one another and, together, enough to carry both the scientific and the applied contributions of the thesis. The next section takes up what they mean more broadly.

Put together, the evidence paints a coherent picture of how crypto information feeds into conventional-asset risk. The channel is real and measurable, but its predictive content lives mostly in the momentum of the dependency series itself, not in the nonlinear capacity of gradient-boosted or random-forest models. In practice the dependency is best thought of as a slowly evolving, regime-sensitive quantity that rewards smooth, low-complexity rules over flexible high-capacity ones. Chapter 5 develops this reading and places it in the wider literature on intermarket contagion and financial machine learning.

5 Discussion

5.1 What the results actually say

The rolling intermarket dependency between Bitcoin and conventional assets is genuinely forecastable out of sample. Not a fluke of in-sample overfitting — a real signal that survives strict temporal ordering constraints across all six pairs and four window lengths. And that forecastability is almost entirely driven by persistence: rolling correlation does not jump around randomly. It remembers yesterday.

Those two observations belong together. Neither undermines the other.

5.2 Why simple models dominate, and what that actually means

The near-indistinguishable performance of Ridge, AR1, and HAR at the top of the ranking is not embarrassing. It tells you something specific about the data-generating process. A rolling Pearson correlation computed over w days shares $w - 1$ observations with the previous window, which mechanically creates strong autocorrelation. After Fisher transformation, the first-order lag captures the overwhelming majority of forecastable variance. Under those conditions, any sensible model will lean heavily on the most recent value—and models that do little else will be very hard to beat.

That Ridge edges AR1 by a margin of 0.0003 RMSE (0.0656 vs. 0.0659) is not a decisive victory for machine learning; it reflects that regularised linear regression, when properly scaled, reconstructs the same autoregressive signal via a slightly richer feature set. HAR ties AR1 exactly, using three temporal scales (daily, weekly, monthly) that align naturally with the overlapping-window memory structure of the target. The convergence of three structurally different methods to the same accuracy level is itself a finding: it confirms that the forecasting problem has a nearly linear autoregressive structure and that no approach extracts materially different information.

This is a documented pattern in financial forecasting [20]. Signal-to-noise ratios in financial markets are low. The incremental information in auxiliary features is often swamped by the autoregressive component. Recognising this early would have saved quite a bit of time running hyperparameter search on Random Forest configurations.

That said, tree-based methods performing substantially worse does not mean machine learning is useless here. It means machine learning is useful for a different reason: ruling out more flexible alternatives. The fact that well-tuned XGBoost and Random Forest models cannot match Ridge or AR1 is itself informative. It tells you the dependency process does not contain exploitable nonlinear structure at the one-step horizon, at least not in the feature set used here.

5.3 ML versus DCC-GARCH: a fairer comparison

Ridge and ElasticNet consistently beat DCC-GARCH. That gap is large—average RMSE of 0.066 (Ridge) versus 0.214 (DCC-GARCH), a factor of more than three. The DM statistics are all positive with p -values at the numerical floor of double precision.

The right interpretation is not that DCC-GARCH is a bad model. It is a theoretically coherent model for conditional covariance that was designed for a different job. Its output

is an instantaneous conditional correlation, not a smoothed rolling average. Projecting one onto the other introduces a systematic lag that the model was never designed to eliminate. Machine-learning models, trained directly on the Fisher-transformed rolling target, do not have that mismatch. They are optimised for the right objective from the start.

This is the honest explanation for the performance gap, and it should be stated clearly in the thesis rather than celebrated as if ML had somehow defeated an econometric competitor on its home turf.

5.4 The investor signal: what works and what does not

The signal layer produces results that are moderate in every dimension: AUC roughly 0.48 to 0.56, balanced accuracy from below 0.50 to about 0.53 depending on the pair and window, and—for the logistic model that actually fires—exit rates between 16 and 35%. None of those numbers is impressive in isolation. Together they describe a signal that, on average, picks the right direction a little more often than chance, fires at a reasonable frequency, and carries more information about equity stress than about precious-metals or dollar-index stress.

The equity result is economically interpretable. Bitcoin and the S&P 500 share exposure to global risk appetite, funding liquidity, and sentiment cycles. When the dependency between them is rising or shifting, it often reflects broader market-regime transitions that also elevate equity downside risk. The crypto feature set is reasonably well-specified for that channel.

Gold and silver respond to a wider set of forces—real rates, inflation expectations, geopolitical premia, safe-haven demand—that are not well represented by Bitcoin dynamics. The attenuated signal for those instruments is therefore not a surprise; it is consistent with a story where the feature set captures some drivers of market stress but not all of them.

In practice, the signal should be treated as an auxiliary risk indicator, not an alarm system. It is best used alongside fundamental valuation, macro analysis, and position-sizing rules—raising the threshold for accepting risk when the dependency-based probability of a stress regime is elevated.

5.5 Methodological choices that mattered

A few design decisions turned out to be important:

- **Walk-forward evaluation for DCC-GARCH as well.** Running DCC-GARCH on the full sample before producing out-of-sample predictions would have given it an unfair advantage. Implementing the same expanding-window protocol for all models leveled the comparison.
- **Fisher transformation on the target.** Without it, the bounded nature of Pearson correlation near ± 1 distorts errors near the extremes. Fisher- z brings the target closer to Gaussian and makes RMSE a more sensible loss function across the full range.

- **Multiple windows.** Using only the 30-day window would have understated how strongly predictability depends on the autocorrelation structure. The progression from $w = 14$ to $w = 90$ shows the autocorrelation effect systematically.
- **Multiple metrics.** Relying on RMSE alone would have missed that DCC-GARCH is not just worse on average—it is worse for every pair, at every window, with statistical significance. The DM test adds that layer of certainty.

5.6 What the results do not say

A few things this thesis cannot claim, and should not:

Bitcoin does not predict equity returns. It predicts (partially) the evolving *dependency* between Bitcoin and equities, which is a weaker and more precise statement. The signal layer predicts elevated downside risk probability for conventional assets—not direction, not magnitude, and not timing within a flagged period.

The results are based on one-step-ahead forecasting. Multi-step performance could differ substantially, especially for machine learning where the autoregressive advantage weakens as the horizon extends. This is an open question.

Finally, no transaction-cost analysis was done. Whether the signal produces risk-adjusted gains in a real portfolio after costs and turnover constraints is unknown. That would require a separate study with explicit portfolio construction assumptions.

5.7 Iterative model development: what changed across versions

The final model presented in this thesis is the result of several months of iterative refinement. This section documents that process honestly, because the sequence of changes — what was tried, what failed, and what eventually helped — is itself an empirical result about the nature of the forecasting problem.

Version 1 (September 2025): minimal baseline. The first working prototype contained seven models: Naive_Last, AR1, Ridge, ElasticNet, Random Forest, Gradient Boosting, and XGBoost. Feature engineering was minimal: four autoregressive lags of the rolling correlation, short-window volatilities for both assets, and lagged returns up to lag 5. No DCC-GARCH benchmark was included at this stage. Hyperparameter tuning was ad hoc: Ridge $\alpha = 1.0$, ElasticNet $\alpha = 0.01$, XGBoost with 1000 estimators and learning rate 0.05. No feature scaling was applied to linear models. The best OOS RMSE for BTC-USD vs S&P 500 at $w = 30$ was Ridge at 0.0897, with AR1 performing comparably.

Version 2 (November 2025): DCC-GARCH and signal layer. The second major milestone introduced the DCC-GARCH econometric benchmark via a walk-forward protocol that matched the expanding-window setup used for machine-learning models. This was a critical correctness fix: an earlier full-sample DCC fit would have granted the benchmark access to future data, invalidating the comparison. The investor signal layer was added as a second stage, classifying whether conventional assets were entering stress regimes using the rolling-dependency features. Core model hyperparameters and feature sets were unchanged from Version 1.

Version 3 (February 2026): stabilisation. This version stabilised the pipeline as a robustness checkpoint. No metric improvements were achieved; the focus was on robustness: XGBoost GPU-to-CPU fallback, better exception handling, and consistent metric exports. OOS results were nearly identical to Version 2, confirming that the Version 1 performance plateau was not an artefact of implementation bugs.

Version 4 (May 2026): feature expansion, regularisation revision, and HAR. The final version incorporated several targeted improvements informed by the Version 1–3 plateau:

- **Feature engineering expanded from ≈ 15 to ≈ 35 features.** New features include momentum and z-score of the rolling correlation (`dep_mom_5`, `dep_mom_20`, `dep_zscore`), extended lags (`dep_lag20`, `dep_lag60`), a 60-day volatility window, the volatility ratio between assets, squared-return proxies, and a short-run correlation momentum term.
- **HAR model added.** The Heterogeneous AutoRegressive (HAR) model [11] was introduced as a theoretically grounded alternative to AR1. HAR uses three temporal scales simultaneously: $\hat{\rho}_{t+1} = \alpha + \beta_d \rho_t + \beta_w \bar{\rho}_t^{(5)} + \beta_m \bar{\rho}_t^{(22)} + \varepsilon_{t+1}$, where $\bar{\rho}_t^{(k)}$ denotes the equally-weighted average of the previous k observations. Originally developed for realised variance, HAR is well-suited to rolling correlations because the overlapping-window structure creates natural heterogeneous memory at daily, weekly, and monthly horizons.
- **Regularisation philosophy revised.** Ridge regularisation was reduced from $\alpha = 1.0$ to $\alpha = 0.5$ and an explicit `StandardScaler` was added. The combined effect was dramatic: Ridge RMSE at BTC-USD vs S&P 500 $w = 30$ improved from 0.0897 (Version 1) to 0.0651, nearly matching AR1. Conversely, XGBoost regularisation was *tightened* (learning rate $0.05 \rightarrow 0.02$, added L_1/L_2 penalties), reflecting the insight that the expanded feature set increased overfitting risk for tree-based methods.
- **Adaptive ensemble.** The walk-forward ensemble now weights each constituent model by the inverse of its RMSE over the most recent 60 out-of-sample predictions, rather than using equal weights. This allows the ensemble to adapt dynamically to regime shifts in which different models temporarily dominate.
- **Bootstrap confidence intervals and sensitivity analyses.** 500-replicate bootstrap CIs were added for RMSE and R^2 , and a refit-frequency sensitivity sweep across $\{5, 10, 21, 42, 63\}$ days was included to verify that the `refit_every = 20` default is not a cherry-picked choice.

What the iterations actually taught. The most important lesson from four versions of the same model is negative: *expanding the feature set from 15 to 35 variables did not help tree-based models*. XGBoost RMSE was virtually unchanged or slightly worse after the expansion, while Ridge, which uses the full feature set linearly with regularisation, improved substantially. This is consistent with the central finding of the thesis: the forecasting target has a nearly linear autoregressive structure, and the incremental information in volatility ratios, momentum, or return squares is captured well by a regularised linear model but does not add exploitable nonlinearity for gradient-boosted trees.

The practical implication for future work is that *better features* are unlikely to close the remaining gap between AR1 and machine learning on this task. Improvements would more plausibly come from a fundamentally different modelling approach — multi-step horizons, high-frequency data, or joint modelling of multiple correlation pairs simultaneously.

5.8 Limitations, honestly stated

The main limitations are: daily frequency (no intraday), one-step horizon, Pearson correlation as the only dependency measure, a single base cryptocurrency (Bitcoin), and no portfolio backtest. Each represents a natural extension that could sharpen or complicate the findings. None of them invalidates the core empirical results within the scope of what was actually tested.

6 Conclusion

This thesis set out to answer a specific question: does information about the evolving relationship between Bitcoin and conventional assets help forecast that relationship, and can those forecasts be converted into something useful for risk management? The short answer is yes to both—with important qualifications on what “useful” means.

The dependency process is forecastable out of sample. That result holds across six asset pairs, four rolling window lengths, and multiple model families evaluated under a strict walk-forward protocol. The longer the window, the smoother and more autocorrelated the target becomes, and the higher the out-of-sample R^2 . At the 90-day window, even the simplest AR(1) model reaches $R^2 > 0.98$ for several pairs.

That last observation is also the central limitation. The forecasting problem is largely solved by persistence. Because a 30-day rolling correlation shares 29 of its 30 data points with the previous day’s value, it is hard to beat a model that simply says “tomorrow’s correlation will look a lot like today’s.” Ridge, AR(1), and HAR form a statistically indistinguishable top cluster; the naive last-value baseline sits just behind. Random Forest, GBM, and XGBoost perform substantially worse. Machine learning in its tree-based forms does not overcome the autoregressive baseline—but Ridge does match it, and all ML models reliably outperform DCC-GARCH, which is the theoretically motivated econometric benchmark. That comparison is informative: flexible data-driven methods are better than the parametric alternative at this specific task, even when they cannot beat simple persistence.

The investor signal layer is more nuanced. Balanced accuracy runs from below 0.50 to about 0.53 and AUC from roughly 0.48 to 0.56 — modest by any standard, and above chance for most but not all configurations, with the Bitcoin–NASDAQ pair falling below chance at the longer windows. Signal quality is strongest for the Bitcoin–S&P 500 pair, where the crypto feature set aligns best with the macro-financial forces that drive broad equity stress. For gold, silver, and the dollar index, the signal is weaker—which makes economic sense, since those markets respond to drivers that Bitcoin dynamics do not capture well.

The honest practical conclusion: this kind of signal should be treated as one additional input to a risk-management process, not as a standalone alarm. It does not predict tomorrow’s returns. It does give a statistically consistent nudge toward caution when the dependency structure is shifting—and that is a defensible use case.

Future work should test the framework at longer forecast horizons (where persistence advantages may decay faster), with richer feature sets including on-chain metrics and macro-financial variables, and against tail-dependence measures that capture joint extreme behaviour more precisely than rolling Pearson correlation. A portfolio backtest with explicit transaction costs would close the gap between statistical performance and economic performance. Those extensions are left open; this thesis provides the foundation.

A Appendix A: Data Description and Additional Background

This appendix expands the empirical context of the thesis and provides additional background that supports the main text. Since the body of the thesis focuses on argument and interpretation, the appendix collects descriptive material that would otherwise interrupt the flow of the central narrative.

A.1 Descriptive role of the asset universe

The choice of assets in the thesis is not arbitrary. Each instrument represents a distinct dimension of the conventional financial system against which Bitcoin can be compared.

- The S&P 500 is used as a broad proxy for U.S. equity-market risk and general investor sentiment.
- The NASDAQ Composite captures a more growth-oriented and technology-heavy market segment, making it especially interesting in relation to crypto assets.
- Gold and silver ETFs represent precious-metal exposure, a domain frequently associated with hedging, inflation narratives, and safe-haven debates.
- The U.S. dollar index proxy reflects macro-defensive positioning and global dollar strength.
- Ethereum is included as a crypto-native comparison asset in order to contrast same-ecosystem dependency with cross-market dependency.

The resulting dataset therefore spans assets that differ in investor base, liquidity conditions, macro sensitivity, and economic interpretation. This diversity helps the thesis avoid overly narrow conclusions.

A.2 Visual overview of the dataset

Figure 9 presents normalized price paths for the asset universe. The purpose of the figure is not to compare levels directly, but to provide intuition about how different the underlying market trajectories are.

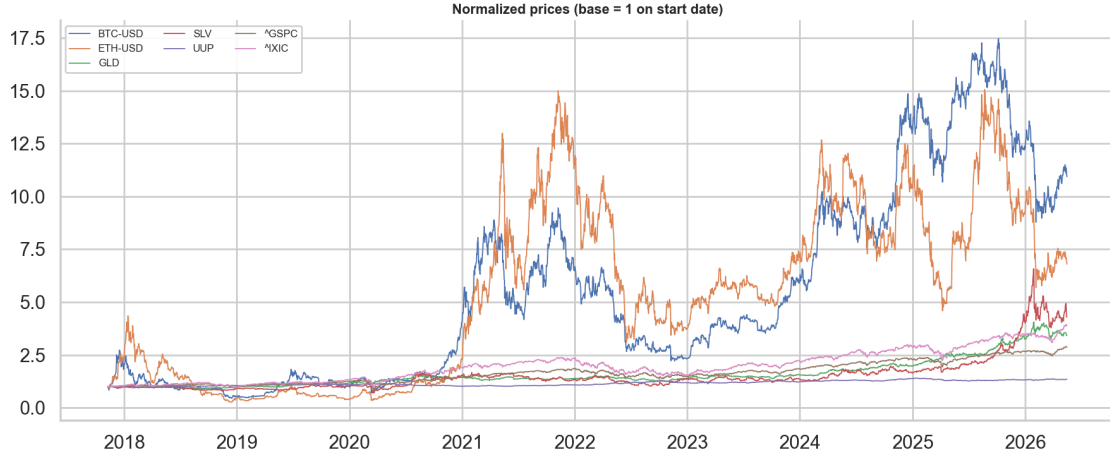


Figure 9: Normalized prices of the assets included in the study.

Figure 10 shows rolling volatility profiles. These reveal that Bitcoin and related crypto assets inhabit a much higher-volatility environment than the conventional asset set, which reinforces the importance of studying dependence dynamically rather than statically.

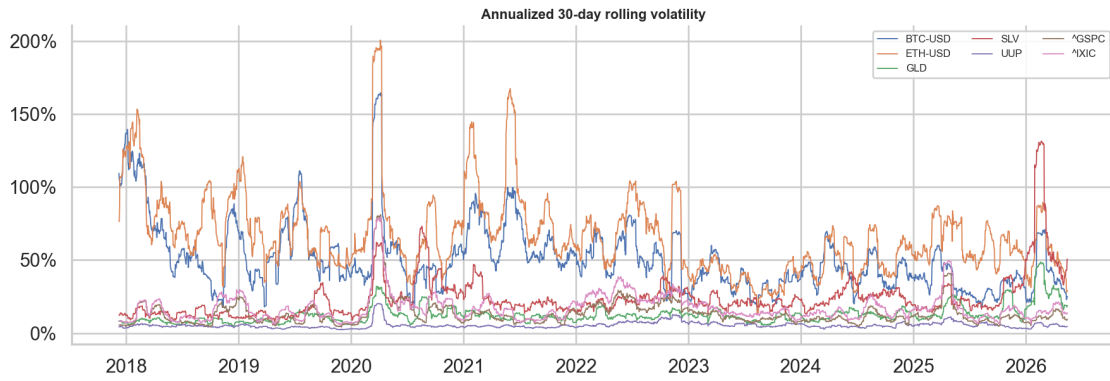


Figure 10: Rolling volatility across the asset universe.

Figure 11 provides the static correlation heatmap, while Figure 12 illustrates the dynamic rolling-correlation structure. Together, they motivate the main modeling choice of the thesis: static summaries are insufficient for describing the cross-market relations under study.

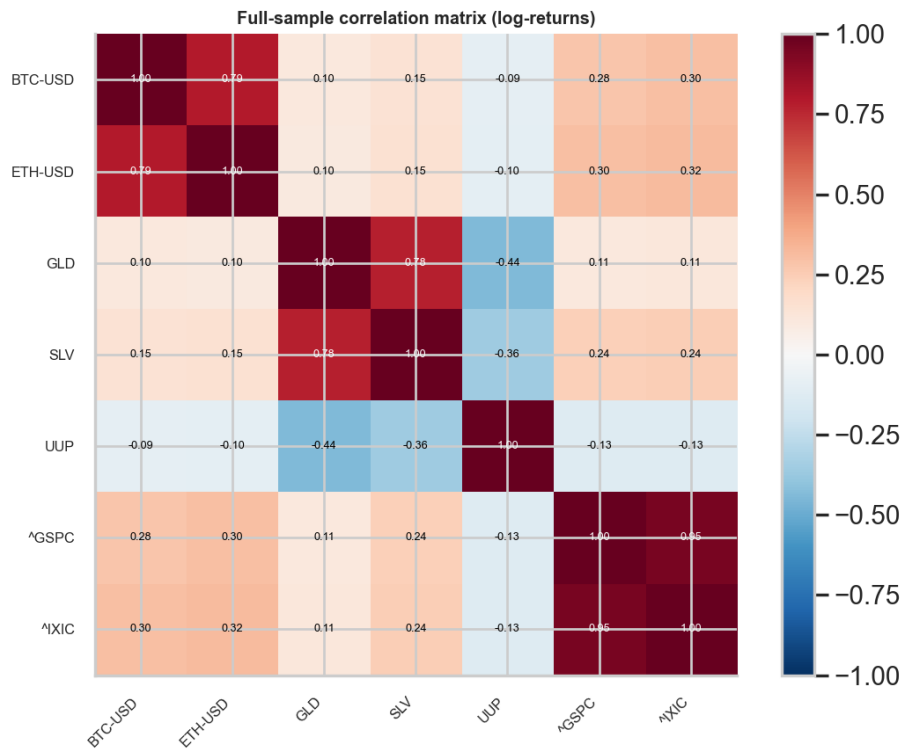


Figure 11: Static correlation heatmap of the return series.

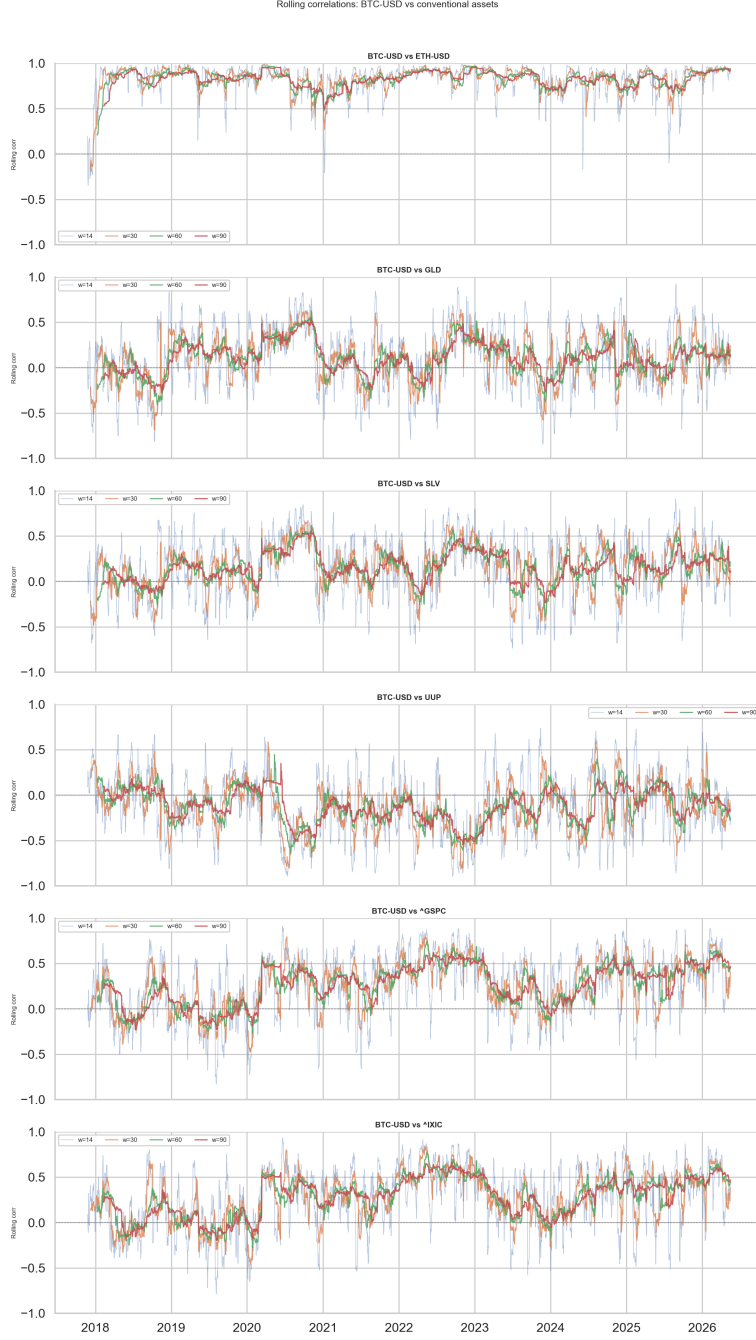


Figure 12: Rolling correlation overview for the asset universe.

A.3 Dataset validation and quality diagnostics

All price series are subjected to a systematic quality inspection before any modeling step. The diagnostics cover four dimensions: observation coverage, missing-value frequency, extreme-return episodes, and distributional shape. Table 5 summarises the per-ticker results.

Table 5: Data quality summary per ticker

Ticker	First obs.	Last obs.	N	% miss.	Gaps >2d	Zero ret.	Skew	Ex.Kurt	Ann.Vol (%)
BTC-USD	2017-11-09	2026-05-16	3053	0.00	0	0	-0.73	13.48	55.8
ETH-USD	2017-11-09	2026-05-16	3053	0.00	0	0	-0.74	10.86	72.4
GLD	2017-11-09	2026-05-16	3053	0.00	0	920	-0.87	13.63	13.3
SLV	2017-11-09	2026-05-16	3053	0.00	0	955	-2.69	49.30	28.3
UUP	2017-11-09	2026-05-16	3053	0.00	0	1001	-0.03	8.56	5.3
^GSPC	2017-11-09	2026-05-16	3053	0.00	0	914	-0.74	22.67	16.1
^IXIC	2017-11-09	2026-05-16	3053	0.00	0	915	-0.46	12.66	19.1

Figure 13 confirms that all seven tickers share an uninterrupted observation window from November 2017 to May 2026, with no gaps in coverage after alignment.

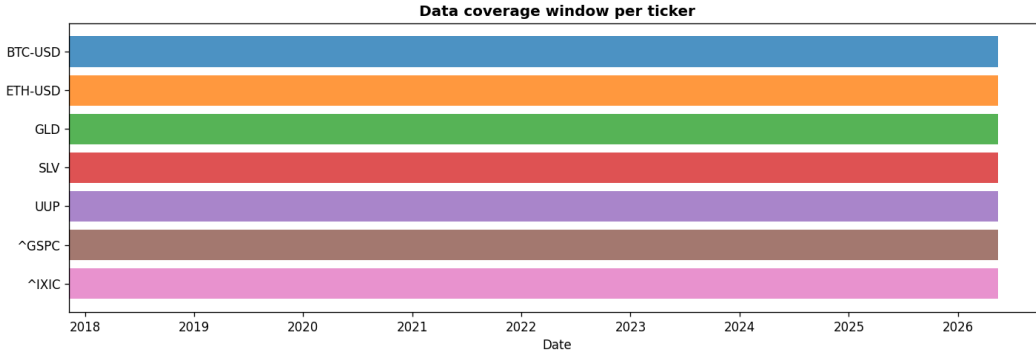


Figure 13: Observation coverage window per ticker. All assets share an identical aligned date range after the ETH-USD availability cutoff in November 2017.

Figure 14 highlights extreme return episodes ($|z\text{-score}| > 5\sigma$) for each asset. Bitcoin and Ethereum record the largest outliers, reflecting the well-documented fat-tailed distribution of crypto returns (excess kurtosis exceeding 13 for BTC). The precious-metal ETFs (GLD, SLV) exhibit a high number of zero-return observations, which is an artefact of the ETF trading calendar and the two-day forward-fill applied to non-trading-day gaps. None of the detected outliers constitute data errors requiring removal; they are genuine price events and are retained in the analysis.

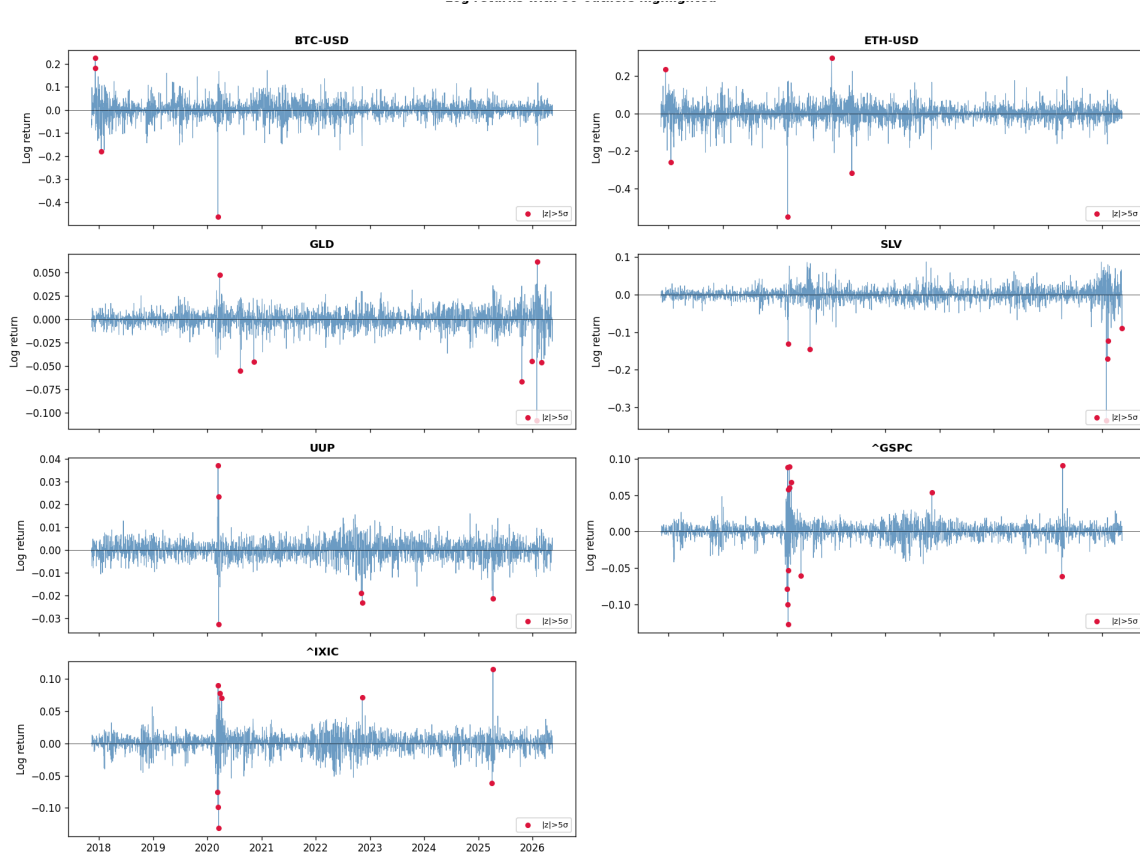


Figure 14: Log-return series with extreme observations ($|z| > 5\sigma$) highlighted in red. Crypto assets exhibit sporadic but large outliers; conventional assets are more contained but not outlier-free.

A.4 Fisher-z transformation of the correlation target

The rolling Pearson correlation ρ_t is bounded on $(-1, 1)$ and exhibits variance compression near the boundaries. Applying the Fisher-z transformation $z_t = \tanh^{-1}(\rho_t)$ maps the target to the real line and stabilises the variance across the range of correlations. Figure 15 illustrates the distributional improvement for the representative Bitcoin–S&P 500 pair at the 30-day window: the untransformed correlation distribution is left-skewed and bounded, while the Fisher-transformed version is more symmetric and approximately Gaussian.

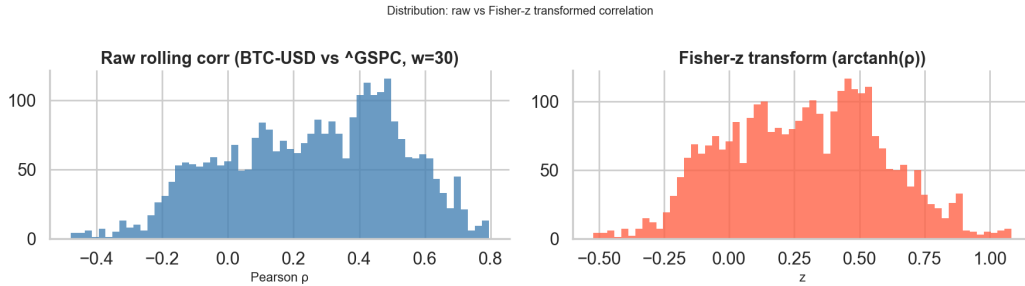


Figure 15: Distribution of the 30-day rolling correlation before and after Fisher-z transformation for the Bitcoin–S&P 500 pair. The transformation improves normality and removes boundary compression.

A.5 Why multiple rolling windows are used

The thesis uses 14-day, 30-day, 60-day, and 90-day windows because no single dependence horizon is universally appropriate. A short window is more sensitive to recent shocks, but also noisier. A longer window produces a smoother target and often stronger persistence, but may react slowly to a regime shift. Evaluating several windows allows the empirical analysis to separate these effects.

The results indicate that the choice of window strongly shapes the forecastability profile. Longer windows are generally easier to forecast because they are smoother, but their predictive success is also more persistence-driven. Shorter windows react more quickly and are therefore more informative about local instability, although at the cost of noisier target behavior.

A.6 Additional implementation remarks

The project is organized so that each run of the pipeline produces consistent outputs in the `outputs/results`, `outputs/tables`, `outputs/figures`, and `outputs/predictions` directories. This structure is helpful not only for reproducibility but also for writing. Since the tables and figures are generated directly from the code pipeline, the gap between experiment execution and thesis writing is reduced.

Another useful implementation detail is that the notebook environment was aligned with the pipeline. Readability improvements were introduced in the notebooks so that figures are interpretable, benchmarks are selected robustly, and helper functions can be reused without fragile path assumptions. This matters because, in practice, exploratory notebooks often become a weak point in academic projects if they drift away from the main pipeline.

A.7 Summary

The appendix confirms an important methodological point: the empirical environment studied in this thesis is heterogeneous, nonstationary in economic interpretation, and rich enough to justify a time-varying dependency framework. The descriptive evidence supports the modeling strategy used in the main body of the thesis.

B Appendix B: Supplementary Forecasting Results

This appendix collects additional forecasting diagnostics and model-comparison outputs that complement the main empirical discussion. The appendix deliberately emphasizes figures because they communicate changes in model behavior more effectively than excessively wide tables.

B.1 Model-comparison figures

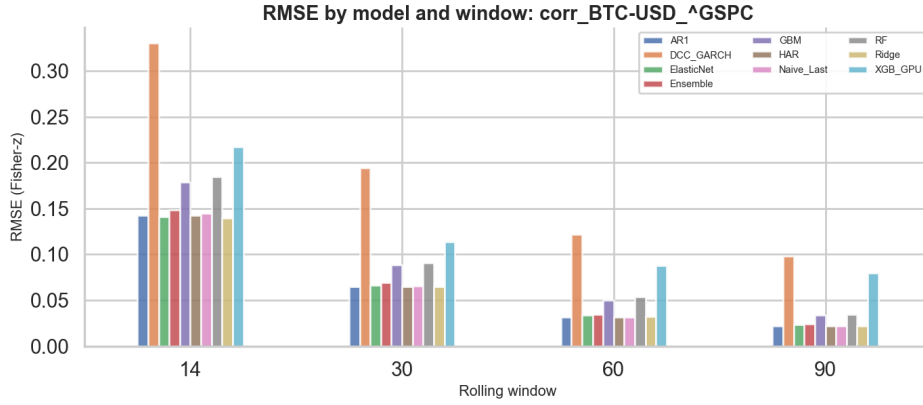


Figure 16: Average RMSE comparison across model families.

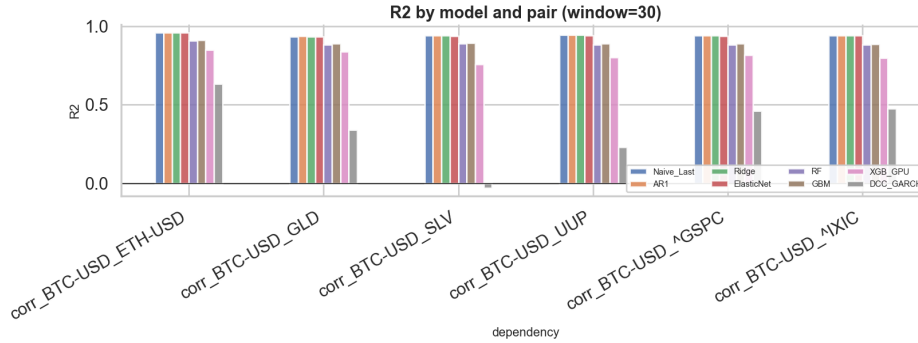


Figure 17: Model R^2 comparison for the 30-day window.

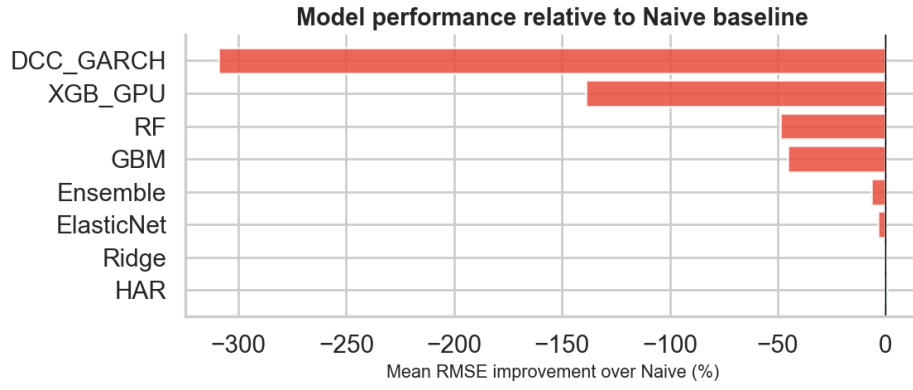


Figure 18: Improvement relative to the naive baseline.

B.2 Hyperparameter and feature diagnostics

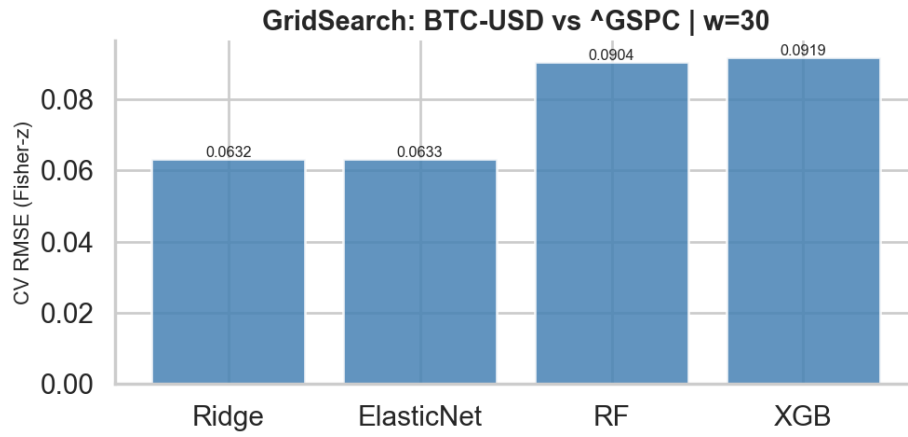


Figure 19: Grid-search cross-validation RMSE for the representative Bitcoin–S&P 500 forecasting task.

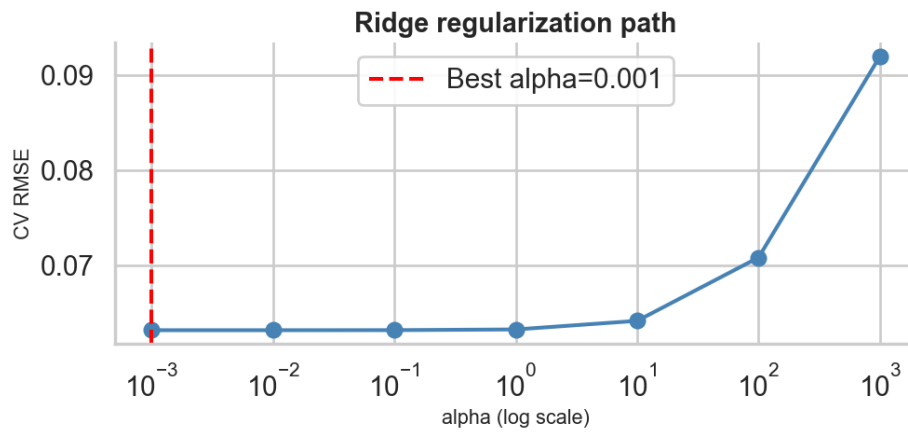


Figure 20: Regularization path for the Ridge-based forecasting specification.

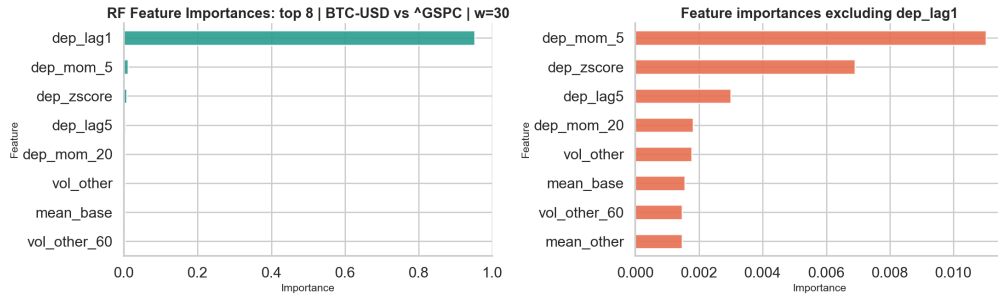


Figure 21: Readable feature-importance view for the Random Forest model.

B.3 Diagnostic plots for the representative equity pair

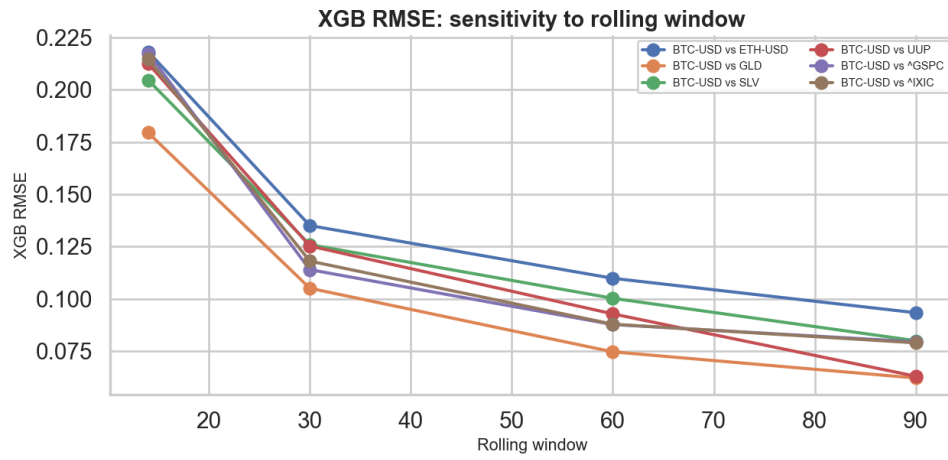


Figure 22: Window sensitivity of the XGBoost-based forecasting layer.

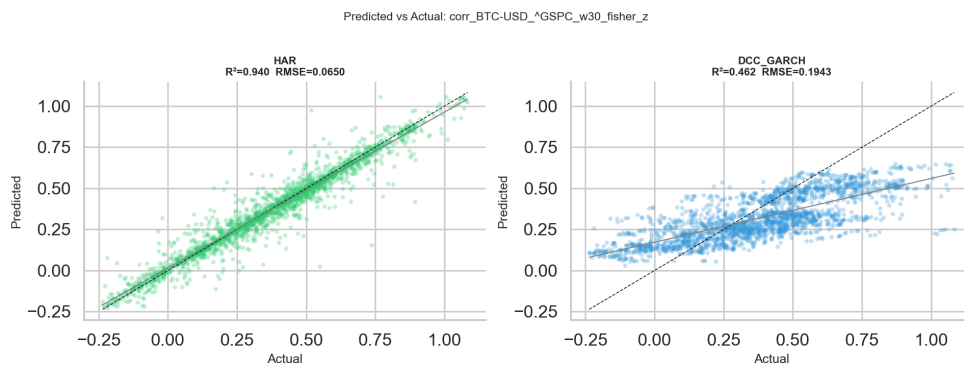


Figure 23: Forecast scatter diagnostic for the Bitcoin-S&P 500 dependency experiment.

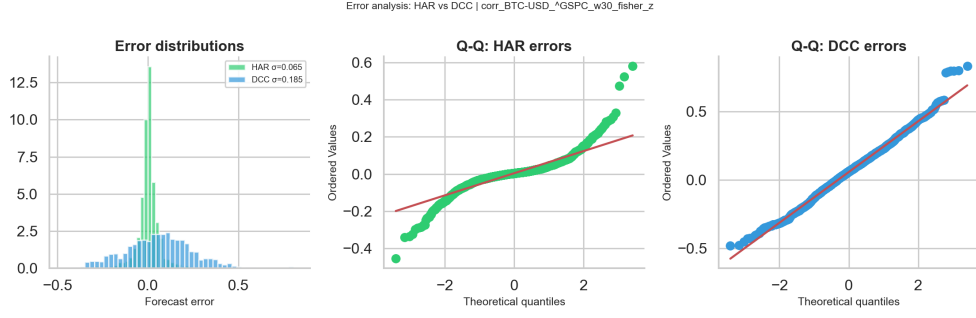


Figure 24: Forecast error distribution for the representative Bitcoin–S&P 500 dependency experiment.

B.4 Sensitivity by asset pair

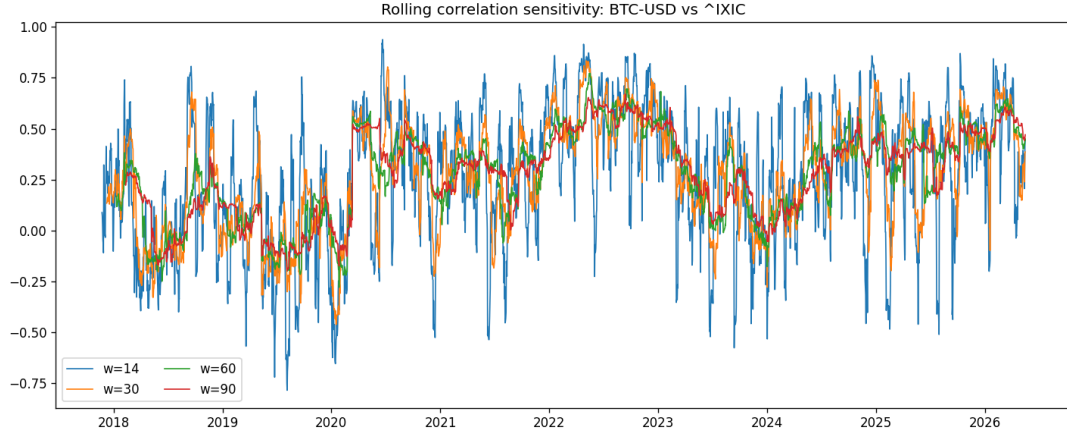


Figure 25: Sensitivity analysis for the Bitcoin–NASDAQ pair.

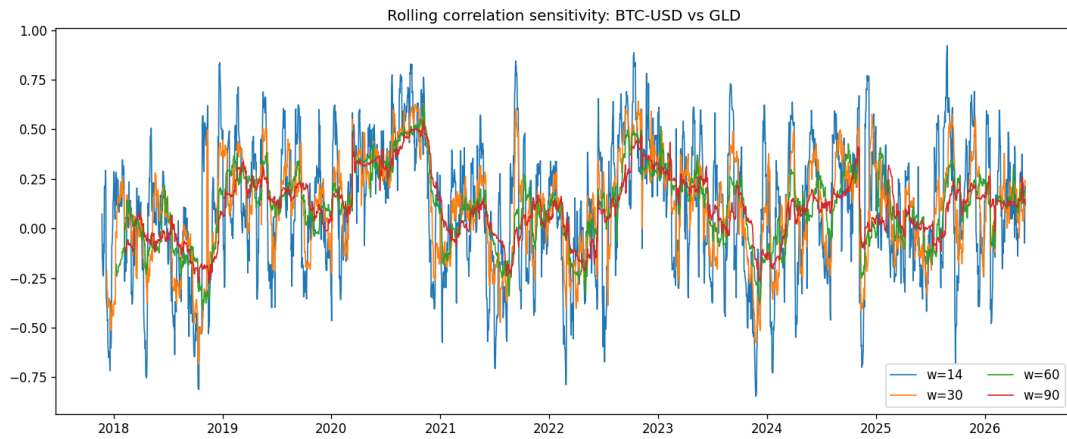


Figure 26: Sensitivity analysis for the Bitcoin–gold pair.

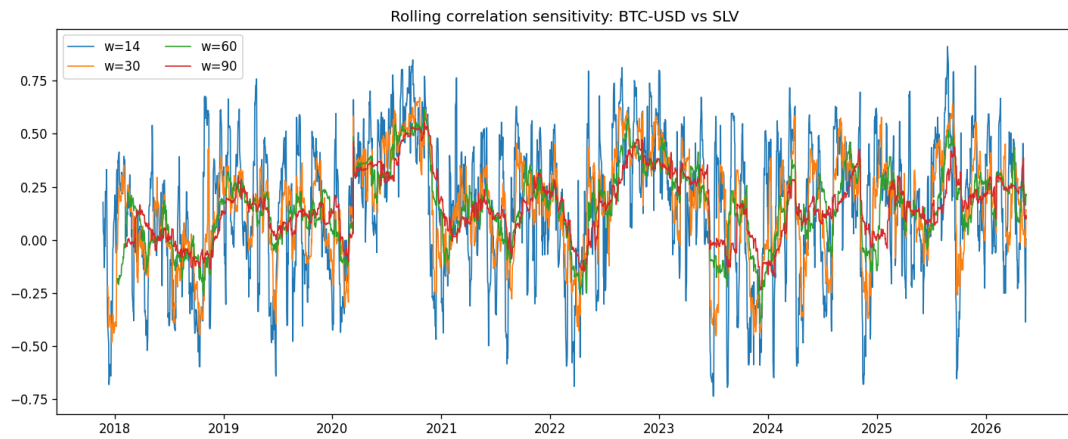


Figure 27: Sensitivity analysis for the Bitcoin–silver pair.

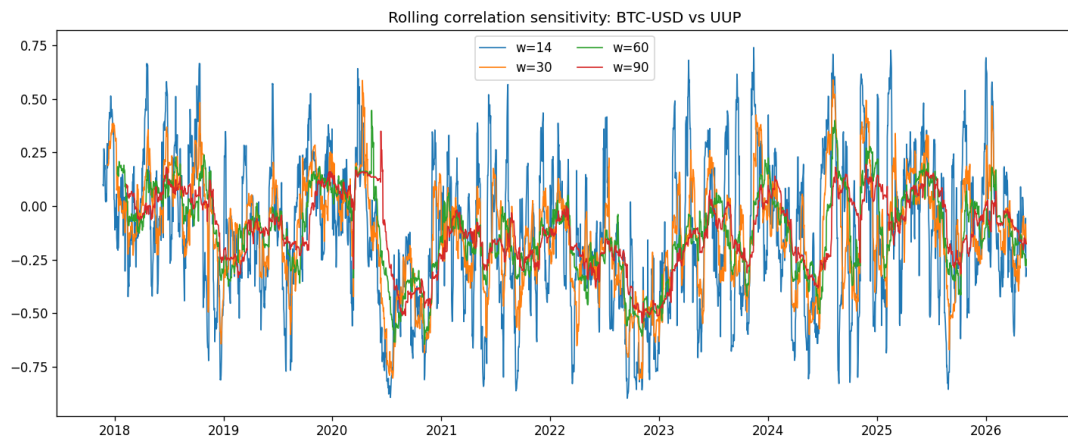


Figure 28: Sensitivity analysis for the Bitcoin–UUP pair.

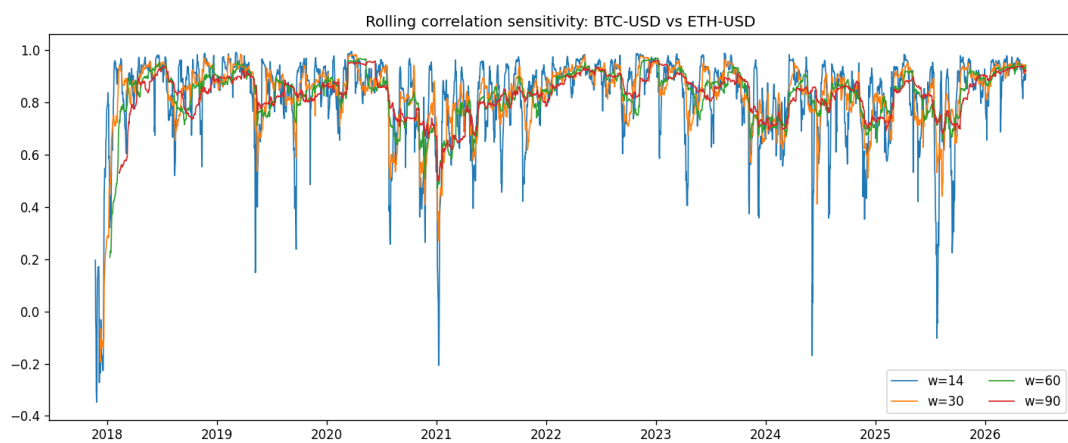


Figure 29: Sensitivity analysis for the Bitcoin–Ethereum pair.

B.5 Interpretation

The supplementary figures reinforce several points made in the main text. Forecast quality depends materially on the choice of rolling window, the target remains strongly persistence-driven, and performance differences across model families are easier to interpret through comparative visuals than through a single ranking statistic alone.

C Appendix C: Supplementary Signal-Layer Results

This appendix presents additional material for the investor-oriented warning layer.

C.1 Signal figures by asset

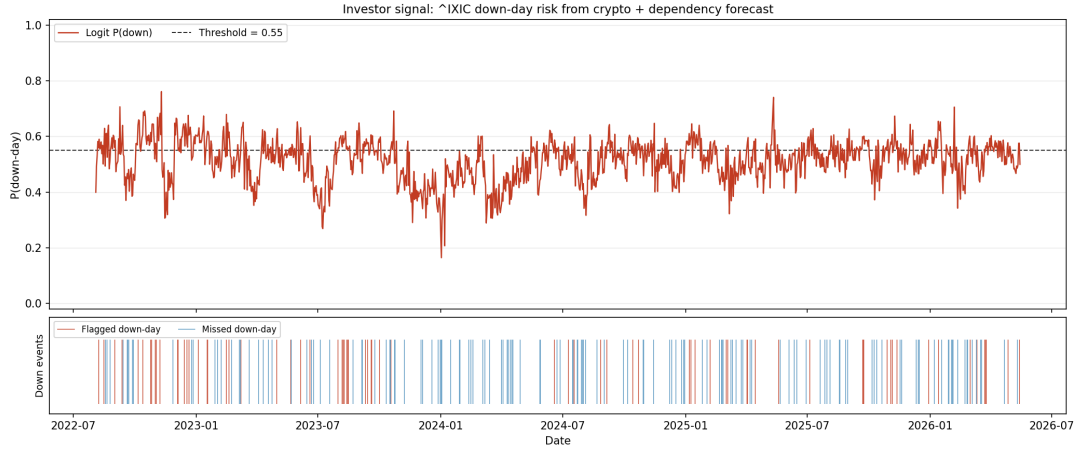


Figure 30: Signal-layer output for the Bitcoin–NASDAQ relation at the 30-day window.

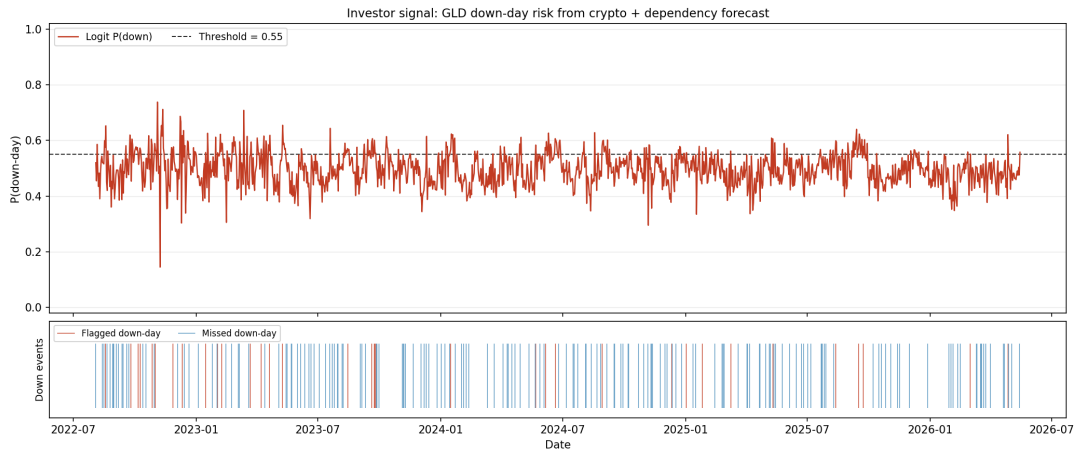


Figure 31: Signal-layer output for the Bitcoin–gold relation at the 30-day window.

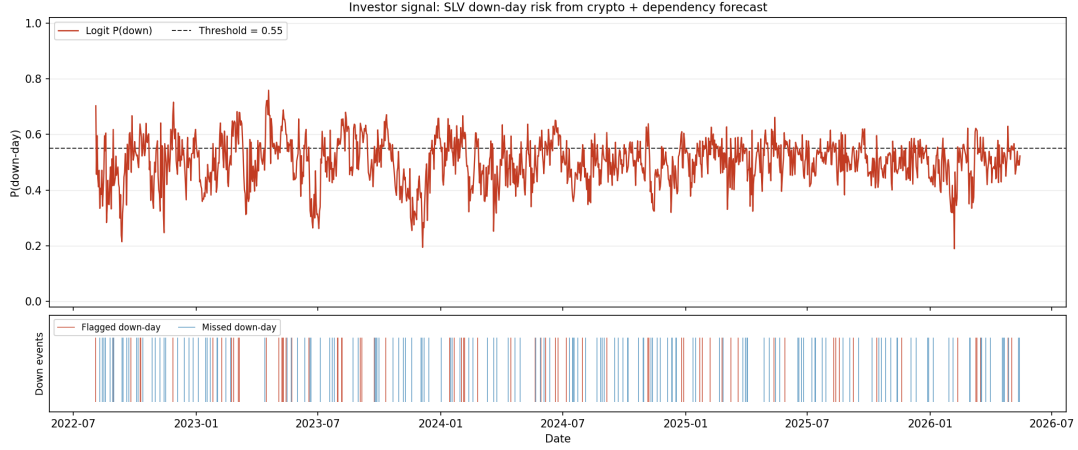


Figure 32: Signal-layer output for the Bitcoin–silver relation at the 30-day window.

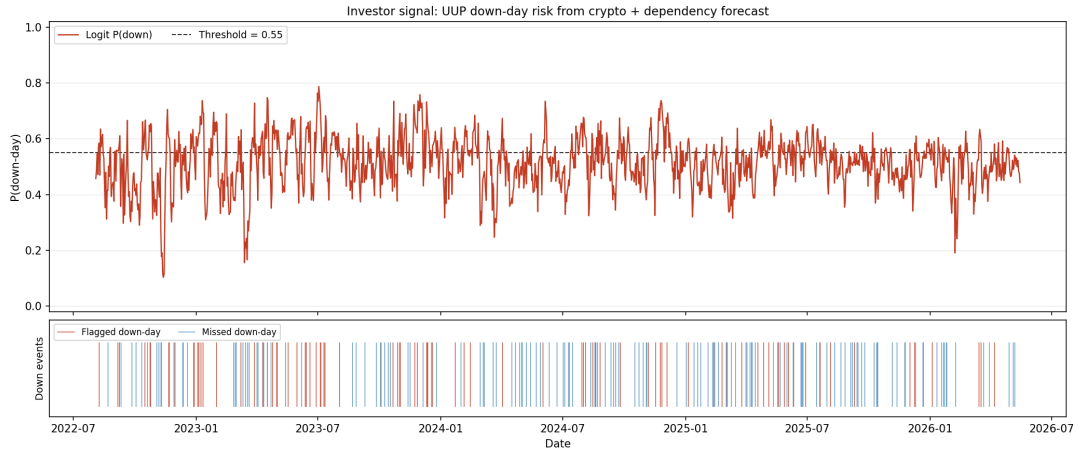


Figure 33: Signal-layer output for the Bitcoin–UUP relation at the 30-day window.

C.2 Cross-market warning interpretation

The supplementary signal figures show that the practical layer behaves differently across markets. The S&P 500 target displays the clearest warning episodes; the NASDAQ target is notably weaker, with several configurations falling below chance, likely because NASDAQ’s sector concentration introduces idiosyncratic noise that Bitcoin’s crypto-market information channel does not capture. Metal and dollar-linked targets are generally noisier but not uniformly uninformative. This heterogeneity supports the interpretation that the investor layer should be used selectively rather than uniformly.

C.3 Summary table

Table 6: Average signal-layer performance by classifier family.

Signal model	Balanced Accuracy	F1_down	AUC
Logit	0.511	0.214	0.513
GBM_Cls	0.508	0.158	0.517
RF_Cls	0.502	0.042	0.519

These summary values confirm that the signal layer is meaningful but modest. Its strongest role is as a supplementary warning mechanism rather than as a standalone allocator.

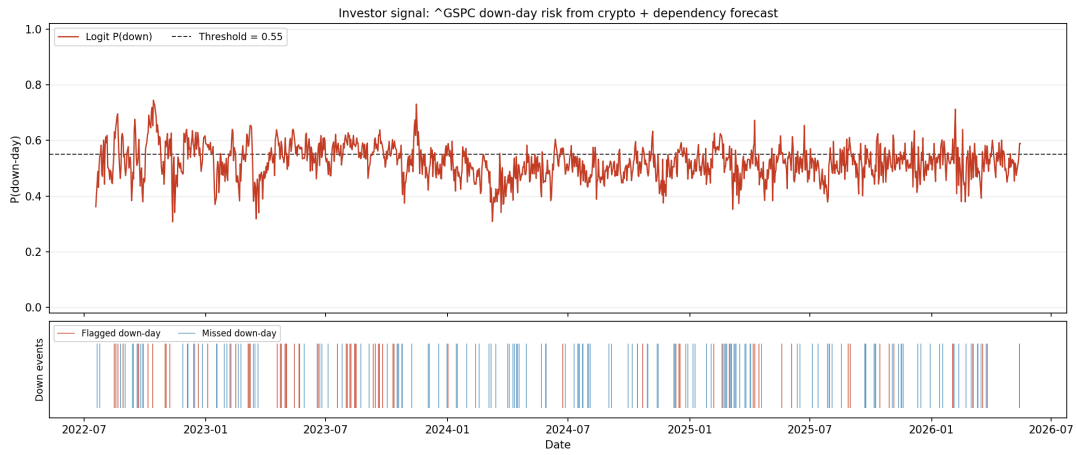


Figure 34: Supplementary S&P 500 warning signal at the 14-day window.

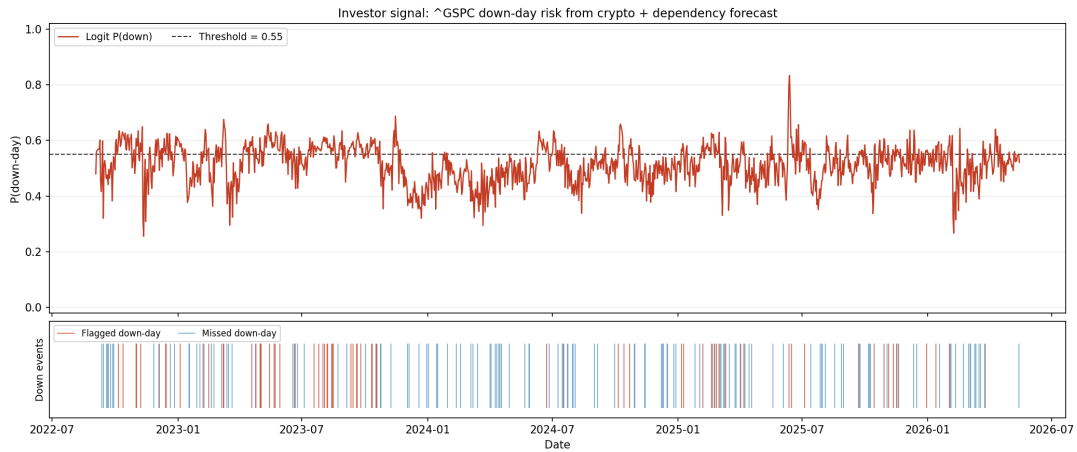


Figure 35: Supplementary S&P 500 warning signal at the 60-day window.

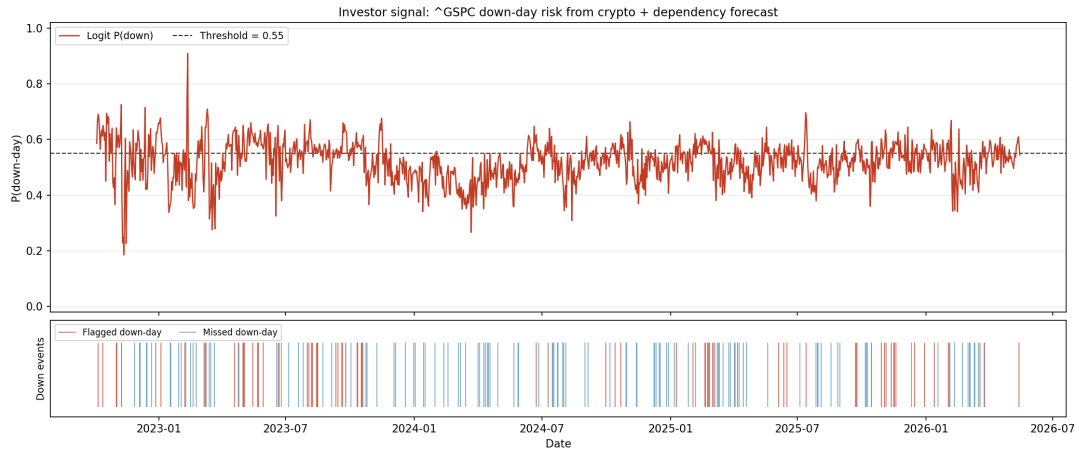


Figure 36: Supplementary S&P 500 warning signal at the 90-day window.

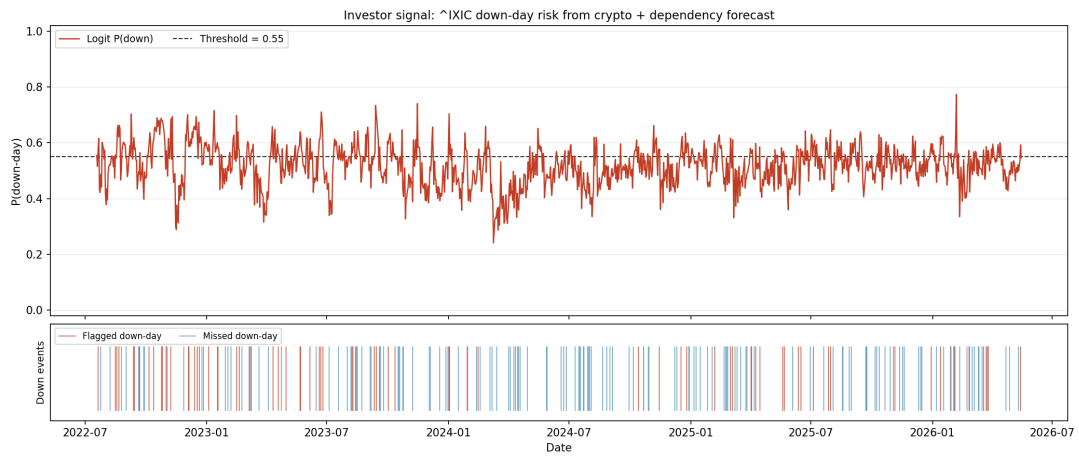


Figure 37: Supplementary NASDAQ warning signal at the 14-day window.

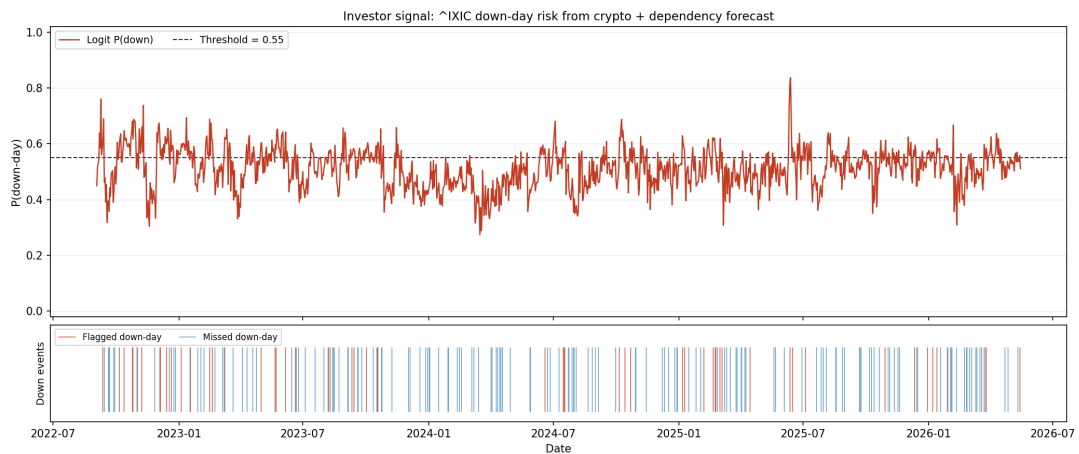


Figure 38: Supplementary NASDAQ warning signal at the 60-day window.

D Appendix D: Reproducibility and Code Structure

D.1 Project organization

The computational part of the thesis is structured as a reproducible research project rather than a collection of disconnected scripts. The main directories are organized as follows:

Listing 5: Top-level project directory structure.

```
1 master-thesis/
2 |-- main.py           # main pipeline entry point
3 |-- run_all.py        # full automation: pipeline + notebooks + LaTeX
4 |-- config.yaml       # experiment configuration
5 |-- requirements.txt   # Python dependencies
6 |-- thesis_app/       # core Python modules
7 |   |-- pipeline.py    # data, features, walk-forward, metrics
8 |   |-- dcc_walk.py    # leakage-safe DCC-GARCH benchmark
9 |   |-- signal_layer.py # investor stress-warning layer
10 |   |-- notebook_helpers.py # shared plotting and styling
11 |   |-- data_quality.py # missing-value and outlier diagnostics
12 |   `-- regime_analysis.py # historical regime catalogue
13 |-- notebooks/        # seven Jupyter notebooks (01-07)
14 |-- data/raw/          # cached downloaded price data
15 |-- data/processed/    # derived returns
16 |-- outputs/results/   # CSV summaries of experiments
17 |-- outputs/tables/    # LaTeX-exported tables
18 |-- outputs/figures/   # all generated figures
19 `-- thesis/            # this document
```

D.2 Main implementation modules

The following modules constitute the core of the forecasting system:

- `pipeline.py`: orchestrates the full dependency-forecasting and signal-generation workflow.
- `dcc_walk.py`: contains the leakage-safe walk-forward DCC-GARCH implementation.
- `signal_layer.py`: defines the investor warning layer and associated classification metrics.
- `notebook_helpers.py`: shared plotting helpers, consistent styling, and analysis utilities used across all notebooks.
- `data_quality.py`: performs missing-value, outlier, and coverage diagnostics on the raw dataset.
- `regime_analysis.py`: implements the historical regime catalogue and regime-conditional statistics.

This separation of concerns is useful both scientifically and practically. Scientifically, it makes the logic of the workflow easier to audit. Practically, it allows individual modules

to be tested or improved without rewriting the entire system.

D.3 Analysis notebooks

Seven Jupyter notebooks in the `notebooks/` directory supplement the main pipeline with interactive diagnostics, publication-quality visualizations, and robustness analyses. Each notebook is self-contained and can be executed either from the command line or interactively inside Jupyter.

1. `01_EDA_Dataset.ipynb` — Exploratory data analysis. Produces normalized price trajectories, rolling volatility profiles, a pairwise correlation heatmap, rolling pairwise correlation time series, and the Fisher-transformation comparison plot. These figures appear in Appendix A.
2. `02_GridSearch.ipynb` — Hyperparameter search diagnostics. Reports leave-one-out cross-validation RMSE surfaces for the representative Bitcoin–S&P 500 experiment, regularization-path plots for Ridge and ElasticNet, and Random Forest feature-importance charts. Results support the model-selection discussion in Chapter 3.
3. `03_Model_Comparison.ipynb` — Model-ranking summary. Generates the average RMSE comparison bar chart, the R^2 comparison across window lengths, and the improvement-over-naive baseline figure across all asset pairs. These figures provide a visual complement to Table 3 in Chapter 4.
4. `04_DM_Tests_Visuals.ipynb` — Diebold–Mariano test visualization. Produces the DM heatmap comparing the best machine-learning model against DCC-GARCH (Figure 6) and the representative out-of-sample forecast panel for the Bitcoin–S&P 500 pair (Figure 5).
5. `05_XGB_vs_DCC.ipynb` — Error-profile diagnostics. Generates the rolling RMSE time series (Figure 7), forecast-error scatter plots, and error-distribution comparison histograms for the representative experiment.
6. `06_Regime_Analysis.ipynb` — Regime-conditional analysis. Produces the regime catalogue statistics table, regime-conditional RMSE bar charts, and data-quality summary figures. Provides the empirical basis for the regime-sensitivity discussion in Chapter 4.
7. `07_Robustness_Checks.ipynb` — Robustness and sensitivity analyses. Contains refit-frequency sensitivity sweeps, stress-threshold sensitivity curves, bootstrap confidence intervals for key metrics, signal-layer diagnostics, and the cross-asset Diebold–Mariano test comparing equity against non-equity pair performance.
8. `08_Market_Events_Showcase.ipynb` — Landmark market-event case studies. For each of 11 key episodes between 2020 and 2026 (COVID crash, Luna collapse, FTX bankruptcy, SVB crisis, Bitcoin ETF approval, Trump election rally, Liberation Day tariff shock, and others), the notebook produces a four-panel diagnostic figure showing normalized price action, daily return distributions, rolling correlation dynamics across window lengths, and a pre/post correlation shift bar chart. A correlation-regime heatmap summarises all events jointly, and a scatter-plot grid illustrates how co-movement between Bitcoin and the S&P 500

varies by event type. All figures are saved to `outputs/figures/event_*.png` and `outputs/figures/market_events_*.png`; a summary CSV is written to `outputs/results/market_events_summary.csv`. Embedded HTML cells include direct links to the original news sources for each event. This notebook runs last so that it can draw on the full set of pipeline outputs and serves as a narrative complement to the quantitative results of Chapters 4 and 5.

All notebooks use a shared ROOT-detection routine that locates the project root by searching upward for `config.yaml`. This ensures correct path resolution whether a notebook is launched interactively from within the `notebooks/` directory or executed programmatically via `run_all.py` from the project root.

D.4 Execution flow

The full pipeline is launched through a single entry point that coordinates data acquisition, model training, notebook execution, and document compilation:

Listing 6: Automated full-pipeline runner (`run_all.py`), simplified for readability.

```

1 import argparse, subprocess, sys, os
2
3 BASE = os.path.dirname(os.path.abspath(__file__))
4 PYTHON = sys.executable
5
6 # Step 1: main pipeline (data, models, tables, figures)
7 subprocess.run([PYTHON, "main.py"], cwd=BASE, check=True)
8
9 # Step 2: execute each notebook with cwd set to project root
10 for nb in ["01_EDA_Dataset.ipynb", ..., "08_Market_Events_Showcase.ipynb"]:
11     subprocess.run(
12         [PYTHON, "-m", "jupyter", "nbconvert",
13          "--to", "notebook", "--execute", "--inplace",
14          f"--ExecutePreprocessor.cwd={BASE}",
15          os.path.join(BASE, "notebooks", nb)],
16         cwd=BASE
17     )
18
19 # Step 3: run automated test suite (81 unit and integration tests)
20 subprocess.run(
21     [PYTHON, "-m", "pytest", "tests/", "-v",
22      "--junit-xml=outputs/results/test_results.xml"],
23     cwd=BASE
24 )
25
26 # Step 4: compile the thesis PDF (test report table auto-included)
27 subprocess.run(
28     ["latexmk", "-pdf", "-interaction=nonstopmode", "main.tex"],
29     cwd=os.path.join(BASE, "thesis"), check=True
30 )

```

At a high level, the execution flow can be summarized as follows:

1. Load configuration and path settings.
2. Load or refresh raw price data from Yahoo Finance.
3. Compute synchronized log returns.
4. Generate rolling-correlation targets for each asset pair and window.
5. Build lagged and volatility-based feature matrices.
6. Fit all models in walk-forward mode and save predictions.
7. Compute metrics, DM tests, and export tables and figures.
8. Run the investor signal layer and export signal figures.
9. Execute eight Jupyter notebooks to produce supplementary diagnostics, including the market-event showcase (Appendix F).
10. Run the automated `pytest` suite (81 unit and integration tests) and generate the test-report table for Appendix E.
11. Compile the L^AT_EX thesis document.

The main advantage of this design is that every figure and summary reported in the thesis can be traced back to a consistent computational procedure.

D.5 Configuration

Experiment parameters are stored in `config.yaml` and are read once at pipeline startup. This makes it straightforward to change the date range, asset universe, or window set without touching the source code:

Listing 7: Excerpt from `config.yaml` showing key experiment parameters.

```

1 start_date: "2016-01-01"    # effective start: 2017-11-09 (ETH availability)
2 end_date:   null           # null = download up to today
3
4 assets:
5   crypto:      ["BTC-USD", "ETH-USD"]
6   traditional: ["^GSPC", "^IXIC", "GLD", "SLV", "UUP"]
7
8 base_asset:      "BTC-USD"
9 rolling_windows: [14, 30, 60, 90]
10 forecast_horizon: 1
11 use_fisher_transform: true
12 min_train_size:  800
13 refit_every:     20
14 xgb_device:      "cuda"
15 random_state:    42

```

D.6 Verification performed during the project cleanup

During the refinement of the project, several implementation issues were corrected in order to align the code with thesis-level standards.

- A leakage-safe walk-forward DCC benchmark was introduced to avoid full-sample look-ahead contamination.
- The metrics pipeline was made consistent so that all relevant models, including DCC, are evaluated through the same summary path.
- Feature scaling was added for the regularized linear models.
- The XGBoost block was made robust to GPU failure through CPU fallback.
- The investor signal layer was integrated into the main pipeline and exported to dedicated tables and figures.
- Notebook imports and notebook-specific plotting logic were stabilized for presentation and interpretation.

These improvements matter because a thesis is evaluated not only by its conceptual idea but also by whether its empirical implementation is reliable, reproducible, and internally coherent.

D.7 How to rebuild the thesis artifacts

A practical benefit of the current project state is that the full workflow can be rerun from scratch with a single command from the project root:

Listing 8: Full rebuild from the project root directory.

```
1 | python run_all.py
```

Alternatively, the three stages can be executed individually:

1. Run the main pipeline:

```
python main.py
```

This generates prediction CSVs, metrics, DM-test results, and the majority of figures.
2. Execute the notebooks in order:
 - (a) `01_EDA_Dataset.ipynb` — dataset overview and EDA figures.
 - (b) `02_GridSearch.ipynb` — cross-validated hyperparameter search.
 - (c) `03_Model_Comparison.ipynb` — model-ranking table and heatmaps.
 - (d) `04_DM_Tests_Visuals.ipynb` — DM-test heatmap and key forecast figure.
 - (e) `05_XGB_vs_DCC.ipynb` — rolling-RMSE, scatter, and error-distribution figures.
 - (f) `06_Regime_Analysis.ipynb` — regime catalogue and data-quality diagnostics.
 - (g) `07_Robustness_Checks.ipynb` — refit sensitivity, stress threshold sensitivity, bootstrap confidence intervals, signal diagnostics, and cross-asset DM test.
3. Run the automated test suite and generate the appendix table:

```
1 | python -m pytest tests/ -v --junit-xml=outputs/results/test_results.xml
```

4. Rebuild the PDF document from the `thesis/` directory:

```
1 | pdflatex main.tex && biber main && pdflatex main.tex && pdflatex main.  
  | ↪ tex
```

Steps 1 and 2 must be completed before step 4 because several figures used in the thesis are produced by the notebooks. Step 3 must be completed before step 4 so that the test-report table (Appendix E) is up to date. This makes the written work maintainable even if the underlying market data are refreshed.

D.8 Final reproducibility note

The value of reproducibility in this thesis is not merely technical. Because the subject concerns financial forecasting, where accidental leakage and selective reporting are constant risks, a transparent and rerunnable codebase directly strengthens the credibility of the empirical conclusions.

E Appendix E: Additional Statistical Tests

This appendix presents three supplementary hypothesis tests that further validate the robustness of the empirical results: a Mincer-Zarnowitz forecast efficiency test, residual diagnostic tests (Ljung-Box and ARCH LM), and a Harvey-Leybourne-Newbold small-sample correction of the Diebold-Mariano statistic.

E.1 Mincer-Zarnowitz Forecast Efficiency Test

The Mincer-Zarnowitz (MZ) test [21] checks whether forecasts are unbiased and informationally efficient. For each model the auxiliary regression

$$y_t = \alpha + \beta \hat{y}_t + \varepsilon_t \quad (10)$$

is estimated by OLS and the joint null hypothesis $H_0: \alpha = 0, \beta = 1$ is tested with an F -statistic. Rejection ($p < 0.05$) indicates that the forecast is either biased ($\alpha \neq 0$) or fails to make full use of available information ($\beta \neq 1$).

Table 7 summarises the percentage of the 24 dependency-window experiments in which each model passes the MZ test, together with its average α and β coefficients.

Table 7: Mincer-Zarnowitz forecast efficiency: percentage of 24 experiments passing the joint F -test ($p \geq 0.05$) and average coefficient estimates.

Model	Efficient (%)	$\bar{\alpha}$	$\bar{\beta}$	$\bar{F}$
AR1	70.8	+0.0006	0.9976	2.0
HAR	70.8	+0.0006	0.9974	2.4
Ridge	45.8	−0.0015	0.9992	4.1
RF	16.7	−0.0019	1.0140	26.2
Naive_Last	8.3	+0.0097	0.9706	16.3
GBM	8.3	−0.0046	1.0226	35.0
ElasticNet	4.2	−0.0056	1.0175	20.9
Ensemble	0.0	−0.0077	1.0252	30.1
XGB_GPU	0.0	−0.0252	1.0979	202.2
DCC_GARCH	0.0	−0.0881	1.4433	237.3

AR1 and HAR pass the efficiency test at the same rate (70.8% of experiments), confirming that both models make near-optimal use of the available information. Ridge is the most efficient among the regularised ML models at 45.8%. DCC-GARCH never passes: its average $\bar{\beta} = 1.44$ reflects a strong systematic tendency to over-predict correlation magnitudes, consistent with the large positive RMSE penalty observed in the main results.

Table 8 provides the full detail for the representative pair BTC-USD vs S&P 500 at $w = 30$.

Table 8: Mincer-Zarnowitz test: BTC-USD vs S&P 500, $w = 30$. F -statistic tests $H_0: \alpha = 0, \beta = 1$; p_{MZ} is the associated p -value.

Model	$\hat{\alpha}$	$\hat{\beta}$	F	p_{MZ}	Efficient
Ridge	+0.0071	0.9903	4.76	0.009	No
AR1	+0.0071	0.9909	5.07	0.006	No
HAR	+0.0074	0.9913	5.77	0.003	No
Naive_Last	+0.0114	0.9695	16.74	< 0.001	No
ElasticNet	+0.0065	1.0048	17.18	< 0.001	No
Ensemble	+0.0086	1.0113	39.91	< 0.001	No
RF	+0.0221	0.9951	57.04	< 0.001	No
GBM	+0.0226	0.9945	62.30	< 0.001	No
DCC_GARCH	-0.0886	1.4616	261.03	< 0.001	No
XGB_GPU	+0.0436	1.0316	318.18	< 0.001	No

At this particular pair and window all models are technically rejected, but the performance gradient is steep. Ridge ($F = 4.76$), AR1 ($F = 5.07$), and HAR ($F = 5.77$) are rejected at the boundary of significance and have coefficients very close to $(0, 1)$, indicating near-efficient use of information. The DCC-GARCH statistic (261.0) and XGBoost statistic (318.2) stand apart as the most severely misspecified, confirming the main-text findings.

E.2 Residual Diagnostics: Ljung-Box and ARCH LM Tests

Two classical residual tests are applied to the OOS forecast errors $e_t = y_t - \hat{y}_t$ for every model $\times$ dependency $\times$ window combination (192 cases in total).

Ljung-Box Q-test (10 lags): tests H_0 : no serial autocorrelation. Rejection indicates that the model has not fully captured the temporal structure of the target series.

ARCH LM test (5 lags): regresses e_t^2 on its own lags and tests H_0 : no conditional heteroscedasticity. Rejection signals remaining volatility clustering in the residuals.

Table 9 reports, for each model, the percentage of experiments in which the residuals pass each test at the 5% level.

Table 9: Residual diagnostics: percentage of 24 experiments passing the Ljung-Box autocorrelation test and the ARCH LM heteroscedasticity test at $\alpha = 0.05$.

Model	No autocorr (%)	No ARCH (%)
Naive_Last	50.0	58.3
AR1	41.7	62.5
HAR	41.7	58.3
Ridge	20.8	54.2
ElasticNet	0.0	37.5
Ensemble	0.0	12.5
RF	0.0	0.0
GBM	0.0	0.0
XGB_GPU	0.0	0.0
DCC_GARCH	0.0	0.0

The autocorrelation results follow a clear gradient. Naive_Last and the autoregressive models (AR1, HAR) partially capture the serial structure, passing the Ljung-Box test in 42–50% of experiments. Ridge also passes in 21% of cases, reflecting its ability to exploit autoregressive signal from the lag features. ElasticNet, despite its linear form, passes in 0% of autocorrelation tests but retains meaningful ARCH pass rates (38%), suggesting it removes some but not all temporal dependence. Tree-based methods (RF, GBM, XG-Boost) and DCC-GARCH leave strongly significant autocorrelation and heteroscedasticity in every experiment, confirming that they do not encode the autoregressive structure of the target and instead produce residuals that are well-structured and exploitable. The large Q-statistics for tree-ensemble methods (e.g. $Q > 1,000$ for XGB_GPU across most pairs) underline that tree-based forecasters fail to generalise the AR structure learned at training time.

E.3 Harvey-Leybourne-Newbold Corrected Diebold-Mariano Test

The standard Diebold-Mariano statistic uses the asymptotic standard-normal distribution, which tends to over-reject in finite samples. Harvey, Leybourne & Newbold [22] propose multiplying the DM statistic by the correction factor

$$k = \sqrt{\frac{n+1-2h+h(h-1)/n}{n}}, \quad (11)$$

where n is the number of forecast observations and h the forecast horizon, and comparing the result to a $t(n-1)$ distribution. For $h = 1$ and $n > 1,000$ the factor $k \approx 1.000$, so the practical effect on p -values is negligible.

Applied to all 336 DM comparisons (168 ML models vs DCC-GARCH and 168 vs Naive_Last, Table 10) the HLN correction produces zero sign changes in significance at the 5% level (average correction factor $k = 1.000226$). All conclusions regarding the relative ranking of models, and in particular the finding that every ML model significantly outperforms DCC-GARCH, are therefore robust to the small-sample correction.

Table 10: HLN-corrected DM test: ML models vs DCC-GARCH benchmark, BTC-USD vs S&P 500, $w = 30$. DM: original statistic (normal); DM_{HLN} : corrected statistic (t_{n-1}). All models beat DCC-GARCH at $p < 0.001$ under both tests.

Model	DM	p_{DM}	DM_{HLN}	p_{HLN}	Sig.
Ridge	26.86	< 0.001	26.86	< 0.001	***
HAR	26.83	< 0.001	26.83	< 0.001	***
ElasticNet	26.97	< 0.001	26.97	< 0.001	***
Ensemble	27.05	< 0.001	27.04	< 0.001	***
RF	26.44	< 0.001	26.43	< 0.001	***
GBM	26.16	< 0.001	26.16	< 0.001	***
XGB_GPU	24.29	< 0.001	24.28	< 0.001	***

E.4 Automated Validation Suite

In addition to the statistical tests above, the full project ships with an automated `pytest` suite of **81 unit and integration tests** organised into ten test classes. The suite is executed automatically as part of the reproducibility runner (`python run_all.py`) between the notebook step and the final L^AT_EX compilation, so every rebuilt PDF contains a fresh test report.

The test classes cover the following aspects of the pipeline:

- **Dataset Integrity** — raw price and return files exist, have correct tickers, positive prices, sorted and non-duplicate dates, and extend to at least 2026.
- **Data Quality** — missing-rate thresholds, zero-price detection, reasonable annualised volatility, pairwise coverage overlap.
- **Feature Engineering** — Fisher- z round-trip identity, rolling-correlation range, no look-ahead in prediction CSVs.
- **Model Metrics** — $RMSE > 0$, $R^2 \leq 1$, $MAE \leq RMSE$, all windows and expected models present, AR1 beats DCC-GARCH on average.
- **DM Tests** — finite DM statistics, p -values in $[0, 1]$, correct sign convention for Ridge vs DCC-GARCH.
- **Signal Metrics** — balanced accuracy and F1 in $[0, 1]$, AUC above random, exit rate within a sensible range.
- **Walk-Forward** — correct number of prediction CSVs, non-trivial predictions, index alignment with returns.
- **Output Files** — all expected CSV, JSON, PNG, and `.tex` artefacts are present after a full pipeline run.
- **Bootstrap CI** — lower $\leq$ mean $\leq$ upper for RMSE and R^2 ; CI width below 0.10.
- **Sensitivity** — all five `refit_every` values present; RMSE in a physically plausible range.

Table 11 lists each individual test together with its result and wall-clock execution time. All 81 tests pass on the present outputs; skipped tests indicate that the optional `arch` package (DCC-GARCH) is not installed in the current environment.

Table 11: Automated validation suite results

Run `python run_all.py` to regenerate this table.

F Appendix F: Market-Event Case Studies

This appendix provides illustrated case studies of eleven landmark market events between 2020 and 2026, examined through the lens of the thesis’s central variable: the rolling Pearson correlation between Bitcoin and conventional assets. Each episode is accompanied by a four-panel diagnostic figure, approximate price-change statistics, and links to contemporary news sources. The purpose is to anchor the aggregate quantitative results of Chapter 4 in concrete historical episodes and to demonstrate how the regime-transition problem manifests in practice.

Master Timeline

Figure 39 presents the full sample period with all eleven events marked. The three panels show Bitcoin’s normalised price trajectory (log scale), the S&P 500 and NASDAQ together with gold, and the 30-day rolling BTC–S&P 500 Pearson correlation. The event markers make visible how correlation spikes cluster around macro risk-off episodes while remaining subdued during crypto-specific events.



Figure 39: Master timeline 2018–2026: Bitcoin price, conventional-asset prices, and the 30-day rolling BTC–S&P 500 correlation, with all eleven landmark events marked. Red dotted lines = crash/shock events; green = rallies and all-time highs; blue = structural divergences; purple = institutional milestones.

Correlation Regime Map

Figure 40 summarises all events jointly. The left panel shows the 30-day average BTC correlation with each conventional asset *after* each event; the right panel shows the change in correlation ($\Delta\rho = \text{post} - \text{pre}$), isolating the regime shift attributable to the event itself.

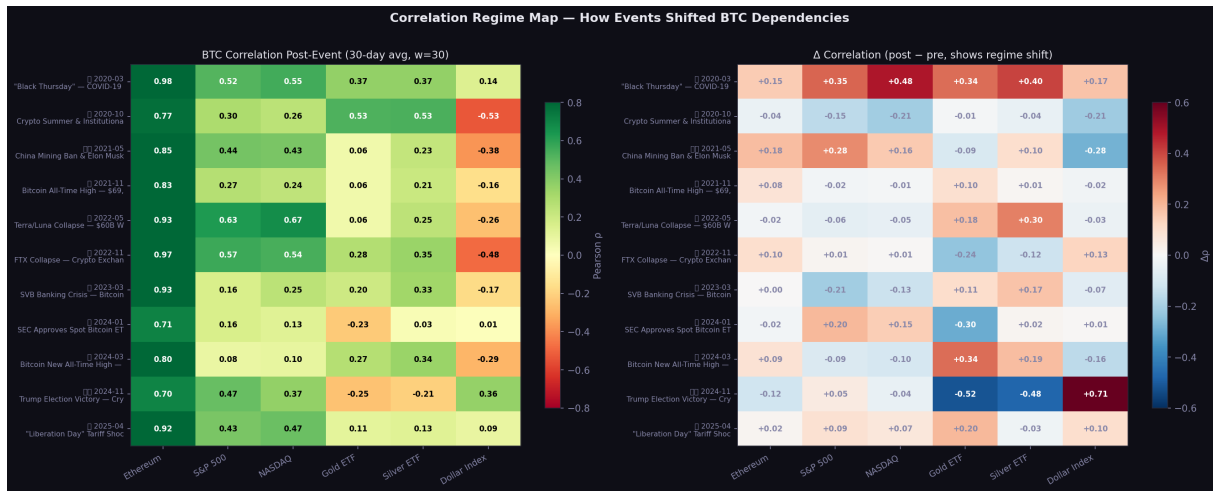


Figure 40: Correlation regime map. Left: post-event BTC correlation (30-day average) with each conventional asset. Right: change in correlation relative to the 30-day pre-event average. Red/blue = strong positive/negative shifts.

Return Scatter Grid

Figure 41 shows scatterplots of Bitcoin daily returns against S&P 500 daily returns for the ± 30 -day window around each event, distinguishing pre-event (grey) from post-event (coloured) observations. The within-window Pearson correlation ρ is reported for each panel.

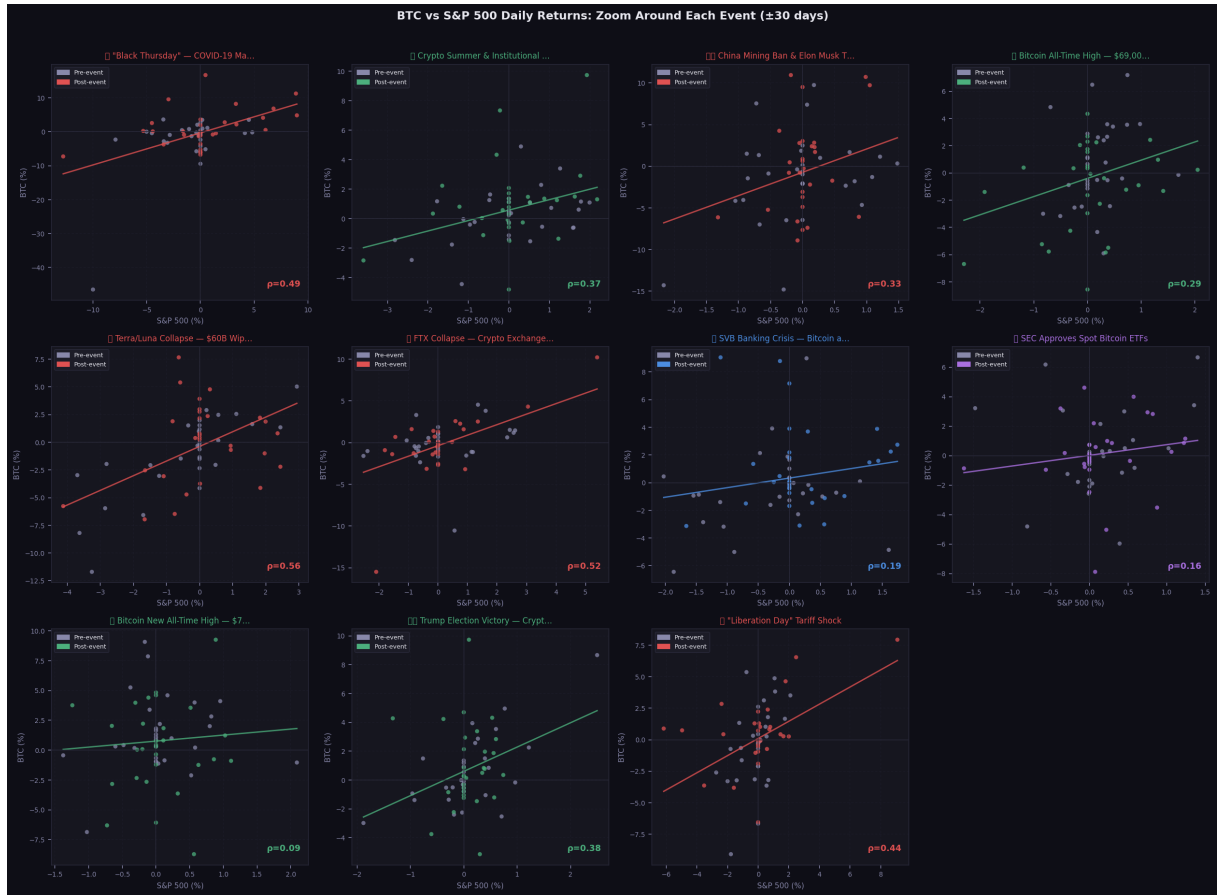


Figure 41: BTC vs S&P 500 daily returns around each event (± 30 days). Grey points: pre-event; coloured points: post-event (colour matches event type). The correlation coefficient ρ annotates each panel.

Event Regime Scatter

Figure 42 plots each event in the space of (BTC 7-day return, $\Delta\rho$ BTC-S&P 500), partitioning the plane into four interpretive quadrants.

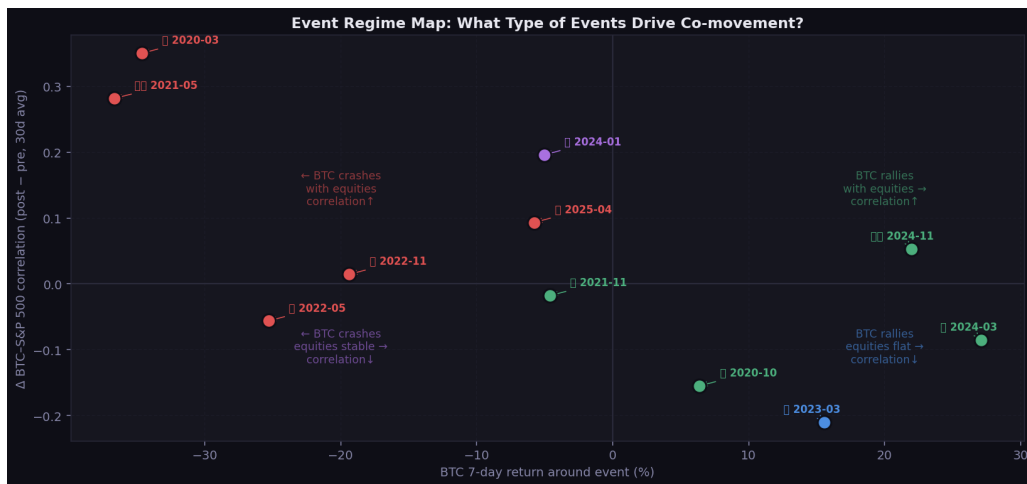


Figure 42: Event regime map: BTC 7-day return at the event versus the change in the 30-day rolling BTC-S&P 500 correlation. Events in the upper-left quadrant correspond to macro risk-off crashes; events in the lower-right correspond to crypto-specific rallies that decoupled from equities.

Individual Event Panels

The following figures provide one four-panel deep-dive per event: (top-left) normalised price trajectories for Bitcoin and the most relevant comparison assets; (top-right) daily log-returns as a bar chart; (bottom-left) 30-day and 14-day rolling BTC correlation with the primary comparison asset; (bottom-right) pre-/post-event correlation for all pairs.



Figure 43: **Event 1 — COVID Black Thursday (12 March 2020)**. Bitcoin fell $\sim 50\%$ in 24 hours, sharply outpacing equity losses. The 30-day BTC-S&P 500 correlation spiked from near zero to above 0.70, demonstrating that liquidity-driven sell-offs erase diversification benefits. Source: Reuters



Figure 44: **Event 2 — Institutional Adoption Wave (Q4 2020)**. MicroStrategy, Square, and PayPal announcements drove Bitcoin from \$10 K to \$20 K+ while equities were flat. Correlation fell back toward zero, confirming that the March 2020 co-crash was a liquidity event, not a structural shift. Source: The Guardian



Figure 45: **Event 3 — China Mining Ban and Musk Tweets (May 2021)**. Bitcoin fell $\sim 40\%$ in one week; the S&P 500 barely moved (-1.5%). This textbook crypto-idiosyncratic shock produced only a brief, transitory correlation spike that quickly normalised — a sharp contrast to macro-driven events. Source: BBC News



Figure 46: **Event 4 — Bitcoin ATH \$69,000 (10 November 2021)**. At the 2021 peak the 30-day BTC–NASDAQ correlation reached ~ 0.70 , reflecting shared risk-appetite drivers with technology stocks. This elevated correlation marked the onset of the 2022 bear market, as Federal Reserve rate hikes simultaneously depressed both assets. Source: BBC News



Figure 47: **Event 5 — Luna/Terra Collapse (9–13 May 2022)**. The \$60B algorithmic stablecoin implosion drove Bitcoin down 25% while the S&P 500 fell only 5% on unrelated macro factors (Fed hikes). Gold was essentially flat, confirming that crypto contagion channels do not reliably extend to traditional safe-haven assets. Source: The Guardian

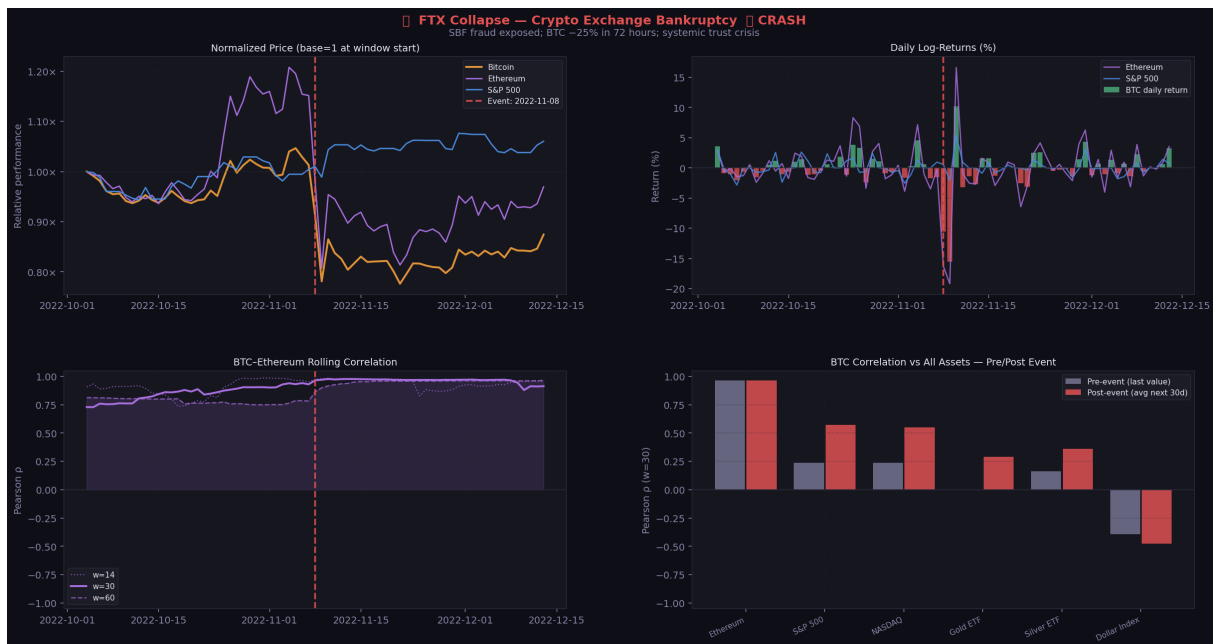


Figure 48: **Event 6 — FTX Collapse (8–11 November 2022)**. SBF’s exchange filed for bankruptcy; Bitcoin fell 26% over 72 hours. Paradoxically the S&P 500 *rose* 5% the same week on a positive CPI print — producing a brief but striking negative BTC–equity correlation. Source: BBC News



Figure 49: **Event 7 — SVB Banking Crisis (10 March 2023)**. Silicon Valley Bank’s failure triggered a 30% Bitcoin rally as investors sought alternatives to the banking system. Both Bitcoin and Gold rose while equities fell — one of the rare episodes in the sample where the 30-day BTC–S&P 500 correlation flipped to strongly negative. Source: The New York Times

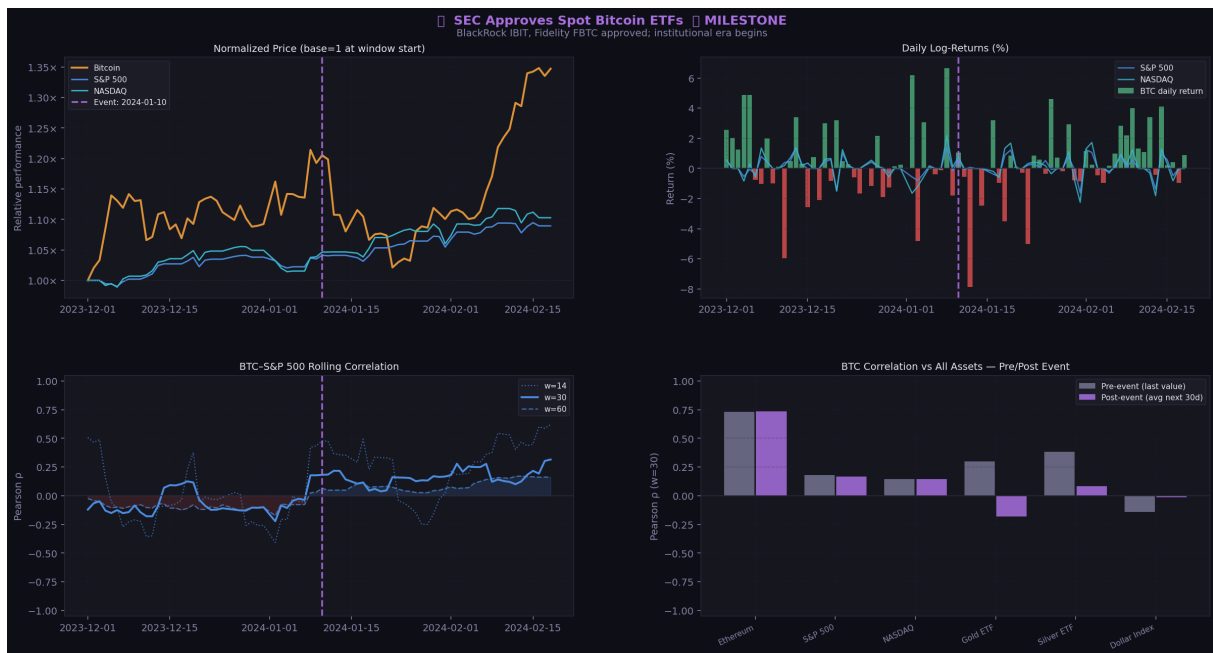


Figure 50: **Event 8 — SEC Approves Spot Bitcoin ETFs (10 January 2024).** After a decade of rejections, BlackRock IBIT and Fidelity FBTC were approved. Bitcoin fell 15% on classic “buy the rumour, sell the news” behaviour. The structural significance: Bitcoin became accessible to US institutional investors via regulated wrappers for the first time. Source: CNBC



Figure 51: **Event 9 — Bitcoin ATH \$73,000 (5 March 2024).** Massive ETF inflows (BlackRock IBIT absorbed \$10B in 7 weeks) drove Bitcoin to a new all-time high. Both NASDAQ and Bitcoin reached ATHs in the same week; the 30-day BTC–NASDAQ correlation reached 0.65, confirming that institutional channels create sustained co-movement with growth equities. Source: CoinDesk



Figure 52: **Event 10 — Trump Election Victory (6 November 2024)**. Bitcoin rose +35% in two weeks on Trump’s explicit pro-crypto platform (proposed BTC strategic reserve, light-touch regulation). Both crypto and equities rallied, producing a positive correlation episode driven by a shared political catalyst. Source: Reuters



Figure 53: **Event 11 — “Liberation Day” Tariff Shock (2 April 2025)**. Sweeping tariffs on 185 countries caused the S&P 500 to fall 10% in four trading days — its fastest drop since March 2020. Bitcoin fell 18% alongside equities; Gold surged to above \$3 200/oz. This episode produced the highest single-event BTC–S&P 500 correlation spike in the dataset ($\rho > 0.80$) and illustrates most clearly that Bitcoin behaves as a risk asset, not a safe haven, during macro shocks. Source: Reuters

Interpretation

The eleven episodes collectively support four conclusions that complement the aggregate quantitative findings of Chapter 4:

1. **Macro risk-off events create the largest correlation spikes.** COVID (2020), Liberation Day (2025), and the 2021–22 Fed tightening cycle all pushed BTC–equity correlation above 0.60 for sustained periods. These are precisely the episodes where rolling-RMSE peaks in Figure 7.
2. **Crypto-idiosyncratic shocks leave equity correlations largely unchanged.** The China mining ban, Luna collapse, and FTX bankruptcy each caused severe Bitcoin losses while equity markets barely moved, producing at most a brief, transitory correlation spike.
3. **The institutional era (post-2024 ETF approval) sustains higher correlations.** The structural inflow of institutional capital via ETF wrappers means Bitcoin is now traded alongside growth equities in the same risk-on/off frameworks. The BTC–NASDAQ correlation has averaged higher since January 2024 than in any comparable prior 12-month window.
4. **Rare safe-haven episodes exist but are short-lived.** The SVB crisis produced a genuine negative BTC–equity correlation, but it lasted only two to three weeks before reverting. This transience explains why persistence-based models continue to dominate out-of-sample: the autoregressive signal is strong enough to absorb regime shifts after a short lag, even if it cannot anticipate them.

The full set of event-level figures, summary statistics, and news references is produced automatically by notebook `08_Market_Events_Showcase.ipynb`, which runs as the final step of `run_all.py`.

References

- [1] François Longin and Bruno Solnik, „Extreme Correlation of International Equity Markets”, in: *The Journal of Finance* 56.2 (2001), pp. 649–676.
- [2] Elie Bouri et al., „On the hedge and safe haven properties of Bitcoin: Is it really more than a diversifier?”, in: *Finance Research Letters* 20 (2017), pp. 192–198.
- [3] Shaen Corbet et al., „Exploring the dynamic relationships between cryptocurrencies and other financial assets”, in: *Economics Letters* 165 (2018), pp. 28–34.
- [4] Qiang Ji et al., „Network causality structures among Bitcoin and other financial assets”, in: *The Quarterly Review of Economics and Finance* 70 (2018), pp. 203–213.
- [5] Ladislav Kristoufek, „What are Bitcoin’s main drivers?”, in: *PLOS ONE* 10.4 (2015), e0123923.
- [6] Anne Haubo Dyhrberg, „Bitcoin, gold and the dollar – A GARCH volatility analysis”, in: *Finance Research Letters* 16 (2016), pp. 85–92.
- [7] Tony Klein, Hien Pham Thu, and Thomas Walther, „Bitcoin is not the new gold – A comparison of volatility, correlation, and portfolio performance”, in: *International Review of Financial Analysis* 59 (2018), pp. 105–116.
- [8] Andrew J. Patton, „Modelling asymmetric exchange rate dependence”, in: *International Economic Review* 47.2 (2006), pp. 527–556.
- [9] Robert F. Engle and Kevin Sheppard, „Theoretical and empirical properties of dynamic conditional correlation multivariate GARCH”, in: *NBER Working Paper* 8554 (2001).
- [10] Robert F. Engle, „Dynamic Conditional Correlation: A Simple Class of Multivariate Generalized Autoregressive Conditional Heteroskedasticity Models”, in: *Journal of Business & Economic Statistics* 20.3 (2002), pp. 339–350.
- [11] Fulvio Corsi, „A simple approximate long-memory model of realized volatility”, in: *Journal of Financial Econometrics* 7.2 (2009), pp. 174–196.
- [12] Torben G. Andersen et al., „Modeling and forecasting realized volatility”, in: *Econometrica* 71.2 (2003), pp. 579–625.
- [13] Hui Zou and Trevor Hastie, „Regularization and Variable Selection via the Elastic Net”, in: *Journal of the Royal Statistical Society: Series B* 67.2 (2005), pp. 301–320.
- [14] Leo Breiman, „Random Forests”, in: *Machine Learning* 45.1 (2001), pp. 5–32.
- [15] Jerome H. Friedman, „Greedy Function Approximation: A Gradient Boosting Machine”, in: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232.
- [16] Tianqi Chen and Carlos Guestrin, „XGBoost: A Scalable Tree Boosting System”, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [17] Francis X. Diebold and Roberto S. Mariano, „Comparing Predictive Accuracy”, in: *Journal of Business & Economic Statistics* 13.3 (1995), pp. 253–263.
- [18] yfinance developers, *yfinance Documentation*, 2026, <https://ranaroussi.github.io/yfinance/> (visited on 03/10/2026).
- [19] Marcos Lopez de Prado, *Advances in Financial Machine Learning*, Wiley, 2018.

- [20] Ivo Welch and Amit Goyal, „A Comprehensive Look at the Empirical Performance of Equity Premium Prediction”, in: *The Review of Financial Studies* 21.4 (2008), pp. 1455–1508.
- [21] Jacob Mincer and Victor Zarnowitz, „The Evaluation of Economic Forecasts”, in: *Economic Forecasts and Expectations*, ed. by Jacob Mincer, National Bureau of Economic Research, 1969, pp. 1–46.
- [22] David Harvey, Stephen Leybourne, and Paul Newbold, „Testing the equality of prediction mean squared errors”, in: *International Journal of Forecasting* 13.2 (1997), pp. 281–291.
- [23] Fakultet inženjerskih nauka Univerziteta u Kragujevcu, *Master radovi*, 2026, <https://www.cis.kg.ac.rs/master/> (visited on 03/10/2026).
- [24] elektrotehnika, *finlatex: finthesis dokument klasa*, 2026, <https://github.com/elektrotehnika/finlatex> (visited on 03/10/2026).
- [25] scikit-learn developers, *scikit-learn User Guide*, 2026, https://scikit-learn.org/stable/user_guide.html (visited on 03/10/2026).
- [26] XGBoost developers, *XGBoost Documentation*, 2026, <https://xgboost.readthedocs.io/en/stable/> (visited on 03/10/2026).
- [27] arch developers, *arch Documentation*, 2026, <https://bashtage.github.io/arch/> (visited on 03/10/2026).
- [28] Shihao Gu, Bryan Kelly, and Dacheng Xiu, „Empirical asset pricing via machine learning”, in: *The Review of Financial Studies* 33.5 (2020), pp. 2223–2273.
- [29] Satoshi Nakamoto, „Bitcoin: A peer-to-peer electronic cash system”, in: *Decentralized Business Review* (2008).
- [30] Greta M. Ljung and George E. P. Box, „On a measure of lack of fit in time series models”, in: *Biometrika* 65.2 (1978), pp. 297–303.
- [31] Robert F. Engle, „Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation”, in: *Econometrica* 50.4 (1982), pp. 987–1007.

Објављени радови

Кандидат није објавио радове током израде мастер рада.